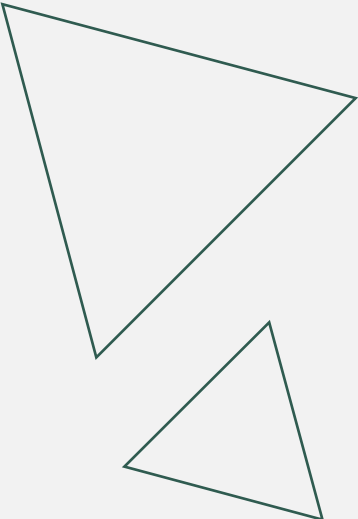


人工智能平台应用场景

讲授人：王子纯

-----2024年4月-----



目录

1

人工智能平台应用概述

- 人工智能平台定义
- 人工智能平台的关键技术

2

人工智能应用典型案例

- 自然语言处理与智能对话
- 图像识别与计算机视觉
- 智能物联网与智能家居
- 智能推荐系统与决策支持

3

人工智能平台开发流程

- 人工智能平台需求分析
- 人工智能平台设计开发
- 人工智能平台测试验证
- 人工智能平台产品交付与运维

4

人工智能平台应用实战

- 容器化部署：*Docker*的安装使用
- 案例：*TrWebOcr*-离线图像识别系统



01



人工智能平台应用概述

- 定义人工智能平台
- 人工智能平台的关键技术

第一节 人工智能平台应用概述

定义

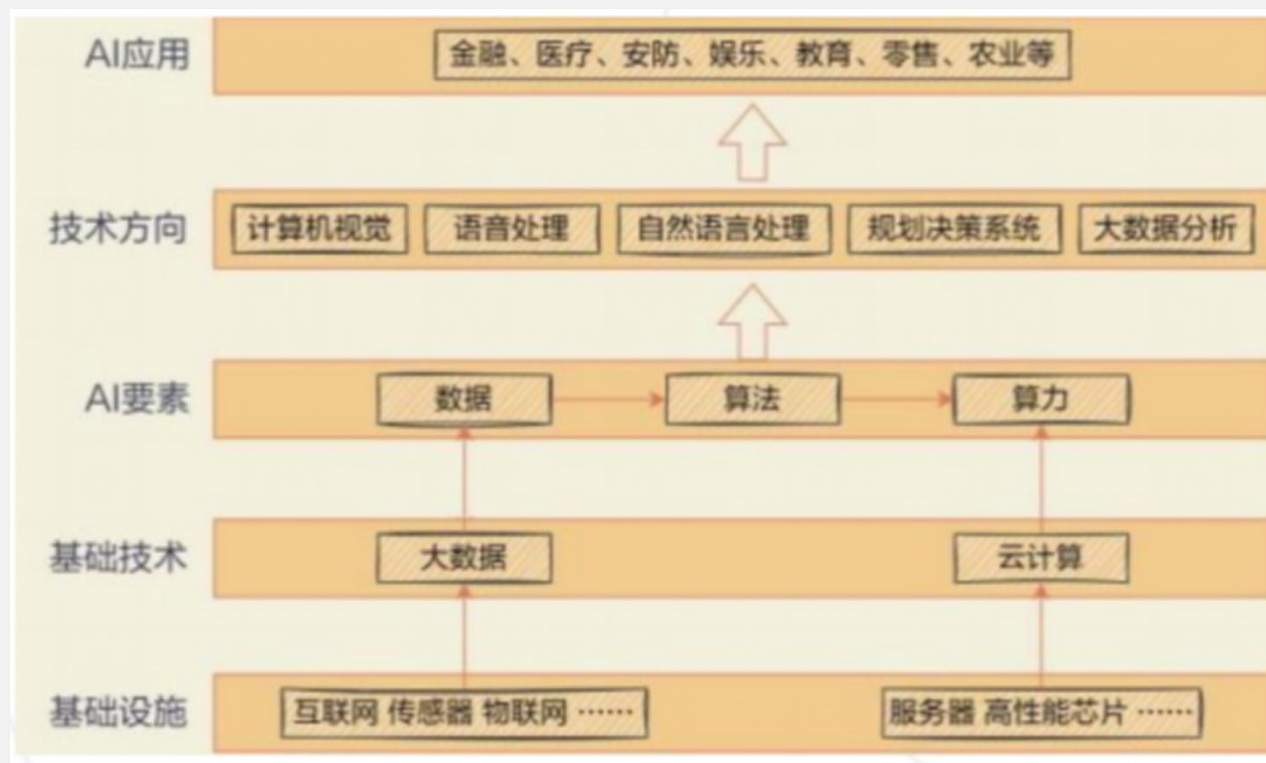
人工智能平台是一种提供人工智能服务和功能的软件系统或框架。它提供了一系列工具、算法和技术，使开发人员能够构建、训练和部署各种人工智能应用。人工智能平台通常包括**数据处理、模型训练、推理推断、自动化部署**等功能，为开发人员提供了一个全面的开发环境。

- 数据处理：人工智能平台通常需要处理大量的数据，包括**数据清洗、特征提取、数据转换**等步骤，以便用于模型训练和预测。
 - 模型训练：人工智能平台提供了各种机器学习和深度学习算法，用于训练模型以从数据中学习并进行预测或决策。
 - 模型推理推断：训练完成的模型可以在人工智能平台上进行推理推断，即使用模型对新数据进行预测或决策。
 - 自动化部署：人工智能平台可以**自动化地**将训练完成的模型部署到生产环境中，以供实时使用。
-
- ✓ 灵活性：人工智能平台通常具有高度的灵活性，可以适应不同类型和规模的人工智能应用开发。
 - ✓ 可扩展性：人工智能平台能够支持不断增长的数据和用户需求，具有良好的可扩展性。
 - ✓ 效率性：人工智能平台能够提高开发和部署人工智能应用的效率，缩短开发周期和降低成本。

第一节 人工智能平台应用概述

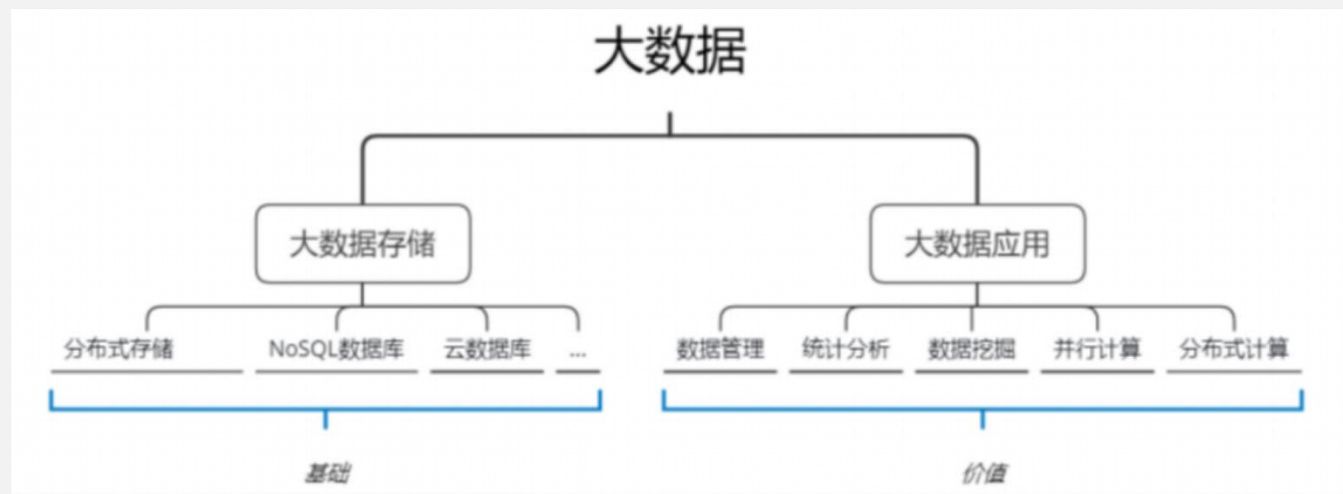
概述

在人工智能平台应用中，基础设施和基础技术提供了支撑 AI 应用的硬件和软件基础，AI 要素和技术方向指导了 AI 应用的开发和应用方向，而行业应用领域则决定了 AI 平台的具体应用场景和业务价值。



第一节 人工智能平台应用概述

大数据存储优化



随着训练数据的规模越来越大，已从以前的TB级别升级到PB级别，使用合理的大数据处理工具进行数据存取、任务编排，才能充分利用数据特征。大数据处理流程主要包括数据收集、数据预处理、数据标注、数据存储、数据处理与分析、数据展示/数据可视化、数据应用等环节。

大数据的分布式处理技术与存储形式、业务数据类型等相关，针对大数据处理的主要计算模型有MapReduce分布式计算框架、分布式内存计算系统、分布式流计算系统等。

第一节 人工智能平台应用概述

机器学习框架



机器学习框架的出现降低了入门的门槛，不需要从复杂的神经网络开始编写代码，可以根据需要使用已有的模型，模型的参数可以通过训练得到，同时可以在已有模型上增加新的Layer，或者是在顶端选择自己需要的分类器。

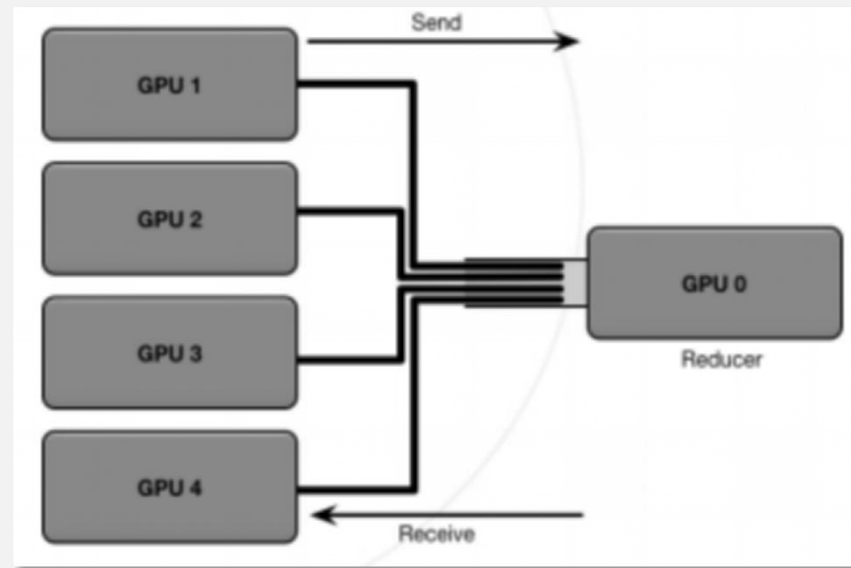
深度学习框架有TensorFlow、PyTorch、SenseParrots等，传统机器学习框架有XGBOOST、SVM-Light等；模型统一表示工具有ONNX、PMML等；模型推理加速框架有TensorRT、TVM等。

第一节 人工智能平台应用概述

高性能计算优化

随着神经网络的参数数量和计算需求的增长，特别是等待数月的大型网络训练，需要在多节点、多智能加速硬件上高效并行加速。

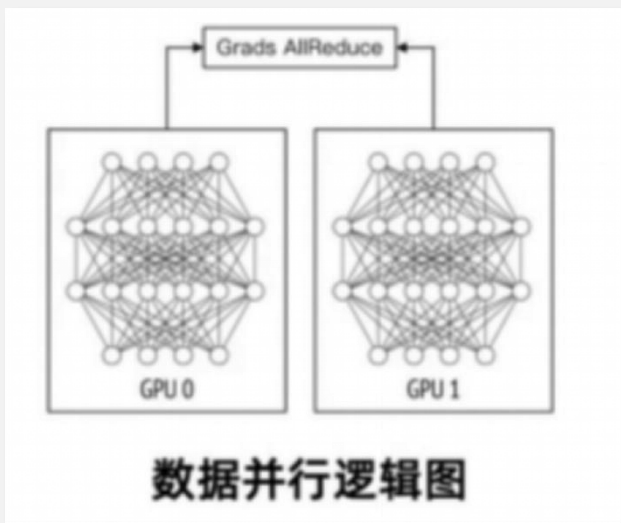
- 并行化策略选择： 在将神经网络的训练并行化到多GPU的过程中，需要选择合适的并行化策略将不同的操作分配到不同GPU上。常见的并行化策略包括数据并行和模型并行。对于大多数情况，数据并行更为常见，因为它能够有效利用多个设备的计算资源，提高训练速度。
- 数据并行优化： 数据并行是将模型部署到多个设备上运行，如多个机器的GPU。
- 模型并行优化： 模型并行是指将模型的不同部分分配到不同的设备上运行，以提高训练的效率。
- 硬件加速优化： 利用高性能计算硬件如GPU、TPU等，以及高速互联网络如InfiniBand等，可以进一步提高神经网络训练的速度和性能。



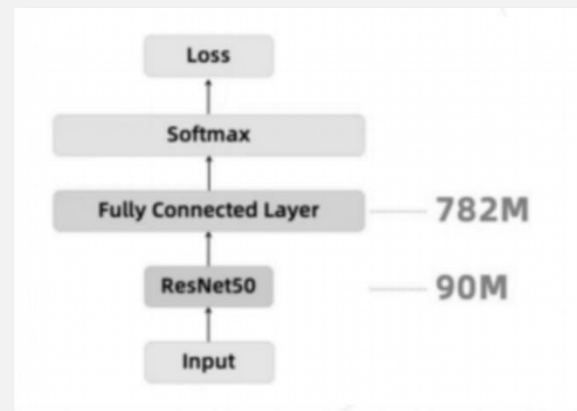
第一节 人工智能平台应用概述

分布式计算优化

数据并行是比较常见的并行化计算逻辑，是将训练数据做打散和分片，然后每张GPU卡取一部分数据进行计算，每张GPU卡保存着全部的模型信息，通过All Reduce算法的方式进行梯度的更新。数据并行经常跟模型并行、流水并行等模式一起使用。



模型并行：当一些模型的部分模块计算量明显高于其它模块的时候，会使用到算子并行的策略。比如Resnet模型做图像分类，当分类器的类别极大，比如要做几十万类别的区分，那么全连接层会变得特别的大。



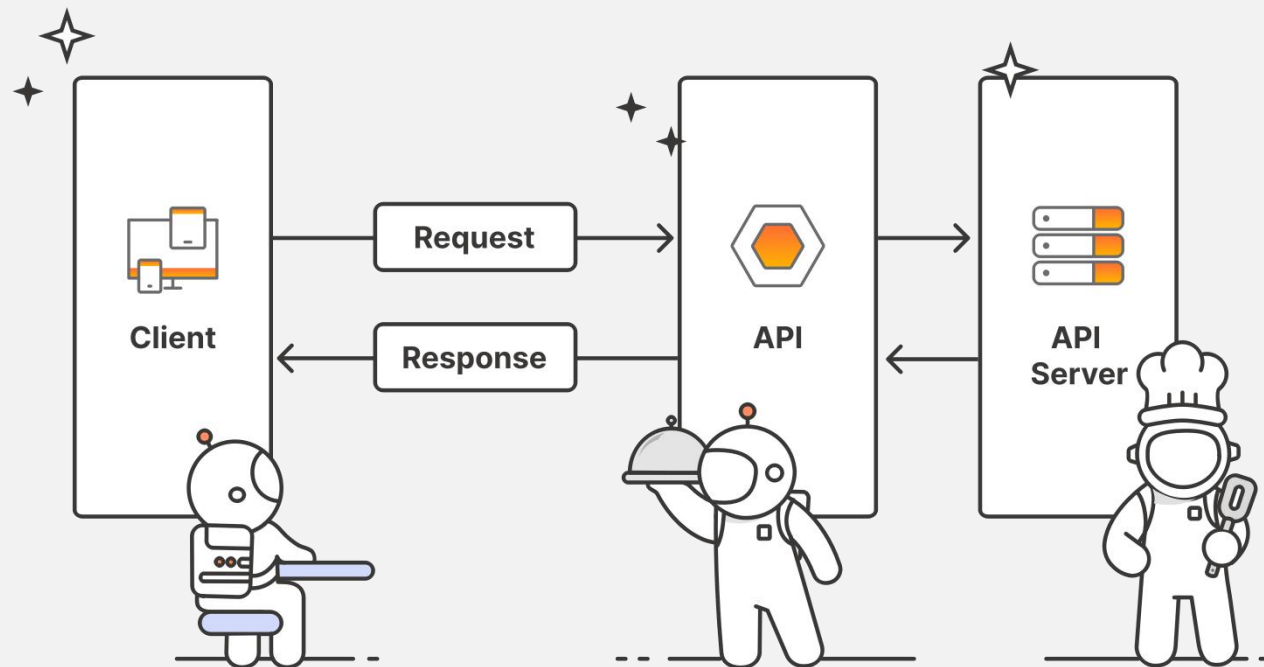
第一节 人工智能平台应用概述

API

人工智能平台产品是深度学习技术应用和发展的重要领域，该领域的主要目标是通过**应用程序编程接口（API）**、**软件开发工具包（SDK）**等方式自动管理全周期人工智能闭环，为上游开发者提供高性能人工智能应用的赋能平台。人工智能平台产品是人工智能不可缺少的重要组成部分。

An **application programming interface** (API) is a way for two or more computer programs or components to communicate with each other. It is a type of software interface, offering a service to other pieces of software.

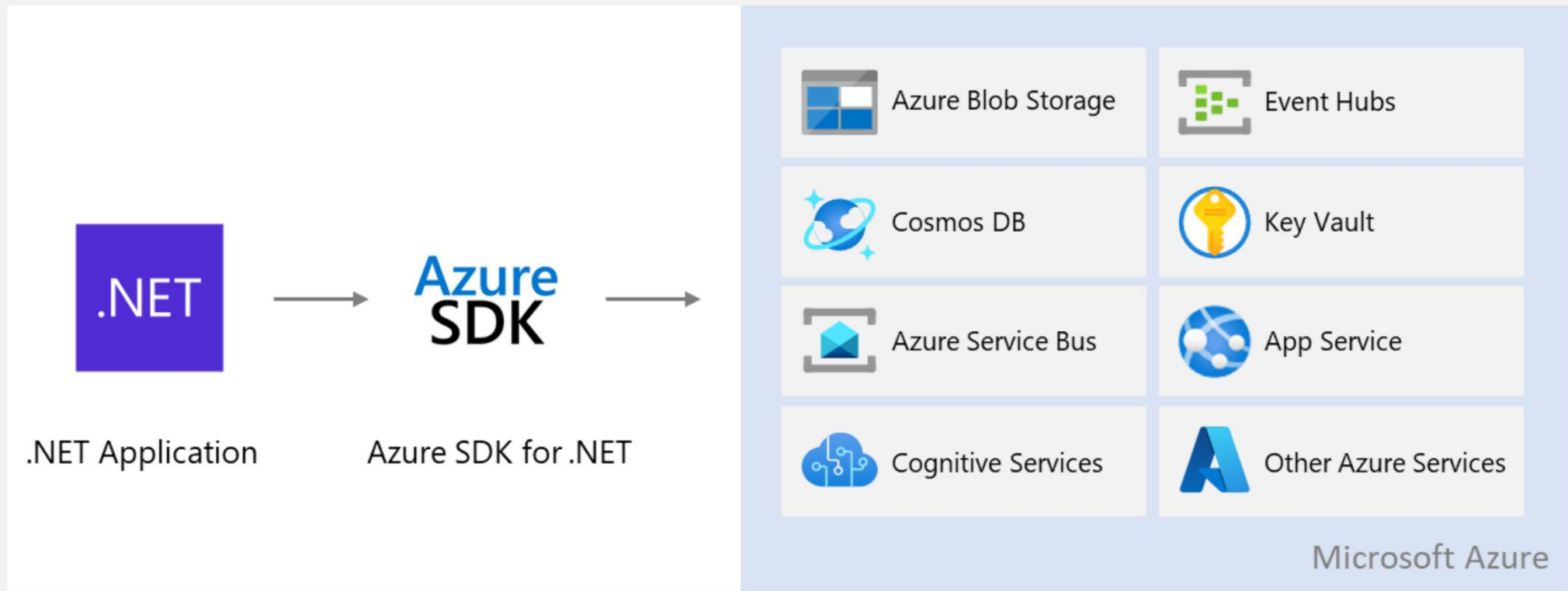
- ✓ 促进系统集成和互操作性
- ✓ 简化开发流程
- ✓ 提高开发效率
- ✓ 促进创新和生态系统发展



第一节 人工智能平台应用概述

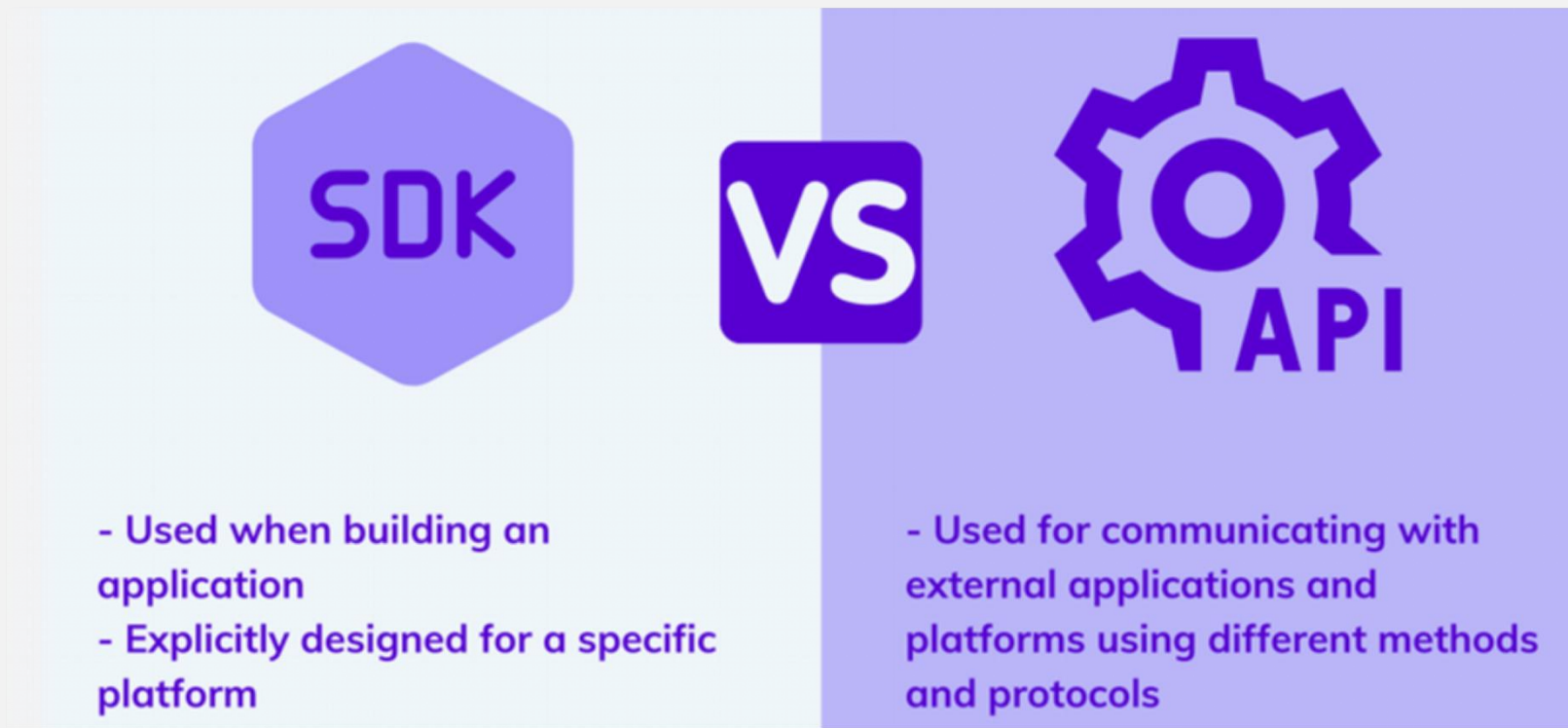
SDK

A **software development kit (SDK)** is a collection of software development tools in one installable package. They facilitate the creation of applications by having a compiler, debugger and sometimes a software framework.



第一节 人工智能平台应用概述

SDK与API的对比



- SDK: SDK更侧重于为开发者提供一套完整的开发环境和工具，以支持应用程序的开发、测试和部署过程。SDK通常包含了编译器、调试器、软件框架等工具，以及示例代码和文档等资源。
- API: API更侧重于定义软件组件之间的交互规范，使不同的软件系统或服务能够共享功能和数据。API不提供完整的开发环境，而是提供了一组标准化的接口和规范，以便开发者能够在自己的开发环境中调用API来实现特定的功能。

02

人工智能应用典型案例

- 自然语言处理与智能对话
- 图像识别与计算机视觉
- 智能物联网与智能家居
- 智能推荐系统与决策支持

第二节 人工智能应用典型案例

AI产品典型应用

- 目前面对行业痛点和问题，企业客户目前都是各人工智能应用独立建设，造成很多竖井式人工智能能力建设项目，资源各自独立，不能复用。且企业内部人工智能建设重复，人工智能资产管理分散，人工智能资产和能力无法得到有效继承，资源浪费较大。建模效率低，不同建模水平的人无法高低搭配。实现敏捷的产业智能应用开发，开发适用于企业实际业务场景的人工智能模型和应用，提升企业实现人工智能化的产业效率，真正将人工智能建设为企业持续创新的核心竞争力。目前各行业的业务大都已全面实现信息化与数字化，也已经在多个业务场景中应用了人工智能技术，人工智能平台产品日益增多，在金融和教育、政府能源行业，探索制造、通信、航空等行业研发中心领域应用。
- 通用人工智能平台通常由六部分组成，分别为：**人工智能中台管控平台、预测服务管理系统、模型开发中心平台、智能数据服务平台、零代码开发平台、模型交付平台。**

第二节 人工智能应用典型案例

AI中台管控平台

- 用户管理（组织、用户、角色）
- 资源管理（机器池、资源池、存储管理）
- 运维管理（监控、审计、警告、日志）
- 系统安全性等参数设置
- 执照管理
- 镜像管理

第二节 人工智能应用典型案例

预测服务管理系统

预测服务系统提供前面的在线服务管理功能，满足**不同模型业务场景需要**。

预测服务支持访问流量控制调度，支持模型多版本之间的**流量配置、切换**，满足持续继承发布的需要。

- 在线服务管理：提供在线服务的创建、配置和管理功能，支持不同模型的部署和服务化，包括模型的部署环境、端口配置、**API**接口定义等。
- 访问流量控制调度：支持对访问流量进行控制和调度，包括流量的负载均衡、流量的限流、流量的调度等，以确保服务的稳定性和可靠性。
- 模型多版本管理：支持对模型的多个版本进行管理和配置，包括模型的版本发布、版本的回滚、版本的切换等，以满足持续集成和发布的需求。
- 流量配置与切换：提供流量配置和切换的功能，支持将流量分配给不同版本的模型，以及在不同版本之间进行流量的切换和调度，以实现模型的灰度发布和**A/B**测试等功能。

模型开发中心平台

模型开发中心平台是一个专注于支持和促进机器学习模型开发和部署的集成平台，主要包括以下功能：

- 模型开发环境：提供**集成开发环境（IDE）**，包括代码编辑器、调试器等工具，以支持模型的开发、测试和调试。
- 模型组件库：提供各种模型组件和算法库，包括常用的机器学习算法、深度学习模型结构、特征工程组件等，以便开发者快速构建和定制化模型。
- 模型版本管理：提供模型版本管理和控制功能，支持模型的迭代和升级，包括模型的**版本控制、版本发布、版本回滚**等操作。
- 模型部署与测试：提供模型部署和测试的功能，支持将模型部署到不同的环境（如**本地、云端、边缘设备**等）并进行测试验证，以确保模型的稳定性和可靠性。
- 模型性能监控：提供模型性能监控和分析功能，包括**模型的运行状态监控、模型性能指标监测、模型日志记录**等，以帮助开发者了解模型的运行情况和性能表现。
- 模型更新与优化：提供模型更新和优化的功能，支持对模型进行进一步优化和改进，包括参数调优、超参数搜索、模型微调等操作。

第二节 人工智能应用典型案例

智能数据服务平台

智能数据服务平台是一个专注于支持人工智能开发过程的平台，主要围绕数据管理、数据清洗和数据标注等功能展开，其主要组成部分包括：

- 数据管理：提供数据的管理和存储功能，包括数据的采集、存储、备份、归档等操作，确保数据的安全、可靠和易于访问。
- 数据清洗：提供数据清洗和预处理的功能，包括数据质量检查、缺失值处理、异常值检测、重复值删除等，以确保数据的准确性和完整性。
- 数据标注：提供数据标注和标记的功能，支持对数据进行人工标注、语义标注、实体标注等，以生成标注数据集，为模型训练和评估提供基础。
- 数据分析与挖掘：提供数据分析和挖掘的功能，支持对数据进行统计分析、可视化分析、关联分析、聚类分析等，以发现数据的潜在规律和价值。
- 数据建模与预测：提供数据建模和预测的功能，支持使用机器学习算法和深度学习模型对数据进行建模和预测分析，以实现对未来趋势和行为的预测。

零代码开发平台

零代码开发平台是为业务开发者、零基础者或追求高效开发者提供便捷高效的建模方式的工具，涵盖图像、视频、文本、语音、结构化数据等五大方向。

- 可视化界面设计：提供直观、用户友好的可视化界面设计工具，使用户可以通过拖拽组件和配置参数来构建应用程序，无需编写复杂的代码。
- 自动化流程生成：提供自动化流程生成的功能，根据用户的需求自动生成应用程序的流程和逻辑，简化了开发过程，减少了开发人员的工作量。
- 快速部署与更新：提供快速部署和更新的功能，支持用户快速部署和更新应用程序，以满足业务需求 and 变化。
- 模型组件库：提供丰富的模型组件库，包括常用的业务组件、数据处理组件、交互组件等，用户可以根据需要选择和配置组件，快速构建应用程序。
- 自动化测试与验证：提供自动化测试和验证的功能，支持对应用程序进行自动化测试和验证，确保应用程序的稳定性和可靠性。
- 灵活扩展与定制：提供灵活的扩展和定制功能，支持用户根据需要定制和扩展应用程序，满足不同业务场景和需求。

第二节 人工智能应用典型案例

模型交付平台

模型交付平台是一个用于集中纳管、评估、优化和转换人工智能模型的处理场所，提供模型纳管、评估、优化、端适配及加密功能。

- 模型集中纳管：提供统一的平台，用于集中管理和存储各种类型的人工智能模型，包括机器学习模型、深度学习模型等。
- 模型评估：提供模型评估的功能，包括模型性能评估、模型准确率评估、模型稳定性评估等，以确保模型的质量和可靠性。
- 模型优化：提供模型优化的功能，包括模型参数调优、模型结构优化、模型压缩等，以提高模型的性能和效率。
- 模型转换：提供模型转换的功能，将模型从一种格式或平台转换为另一种格式或平台，以适配不同的硬件设备或应用场景。
- 部署管理：提供模型部署和管理的功能，包括模型部署到不同环境（如云端、边缘设备等）、模型版本管理、模型监控等。
- 自动化流程：提供自动化的流程和工作流，支持模型交付过程的自动化和标准化，减少人工干预和错误。
- 安全保障：提供安全保障措施，包括数据加密、访问控制、身份认证等，确保模型和数据的安全性和隐私保护。

第二节 人工智能应用典型案例

典型AI平台架构

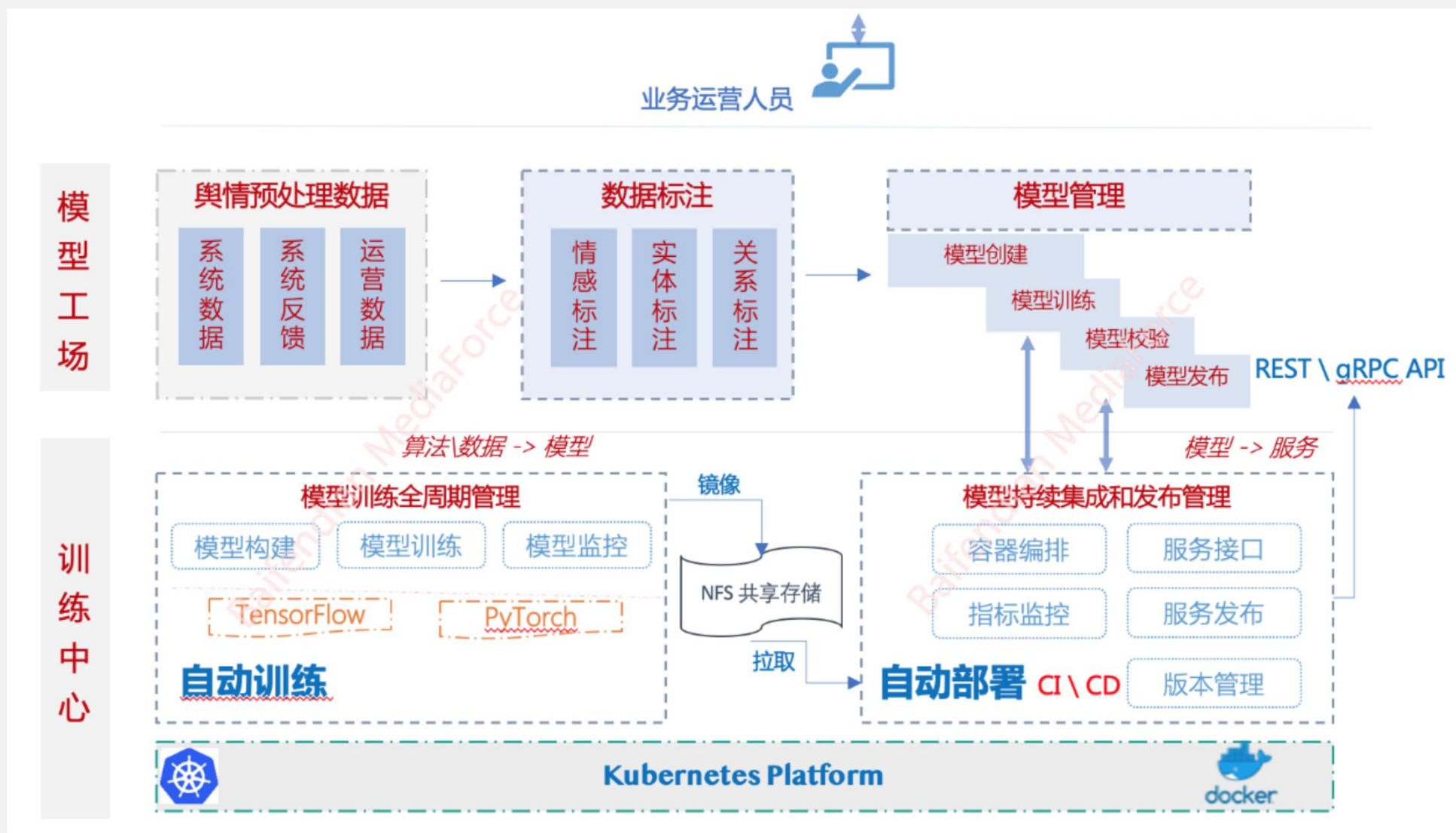
来源：百度智能云 - 能源AI中台解决方案



第二节 人工智能应用典型案例

典型AI平台架构

来源：百分点大数据技术团队：舆情平台架构实践与演进

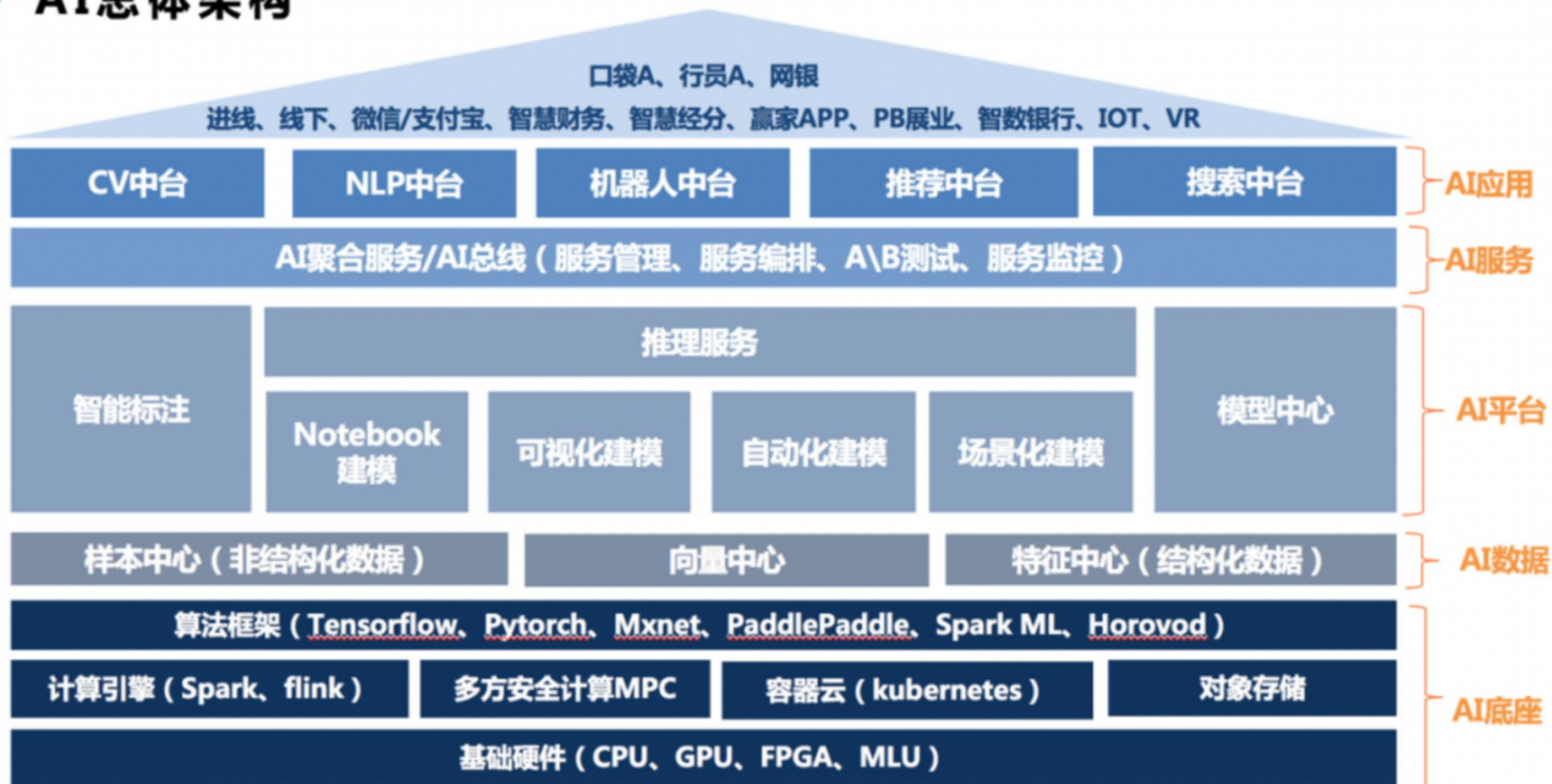


第二节 人工智能应用典型案例

典型AI平台架构

来源：金融智能化转型 - 平安银行AI中台架构搭建实战

AI总体架构



第二节 人工智能应用典型案例

人工智能工程技术职业发展

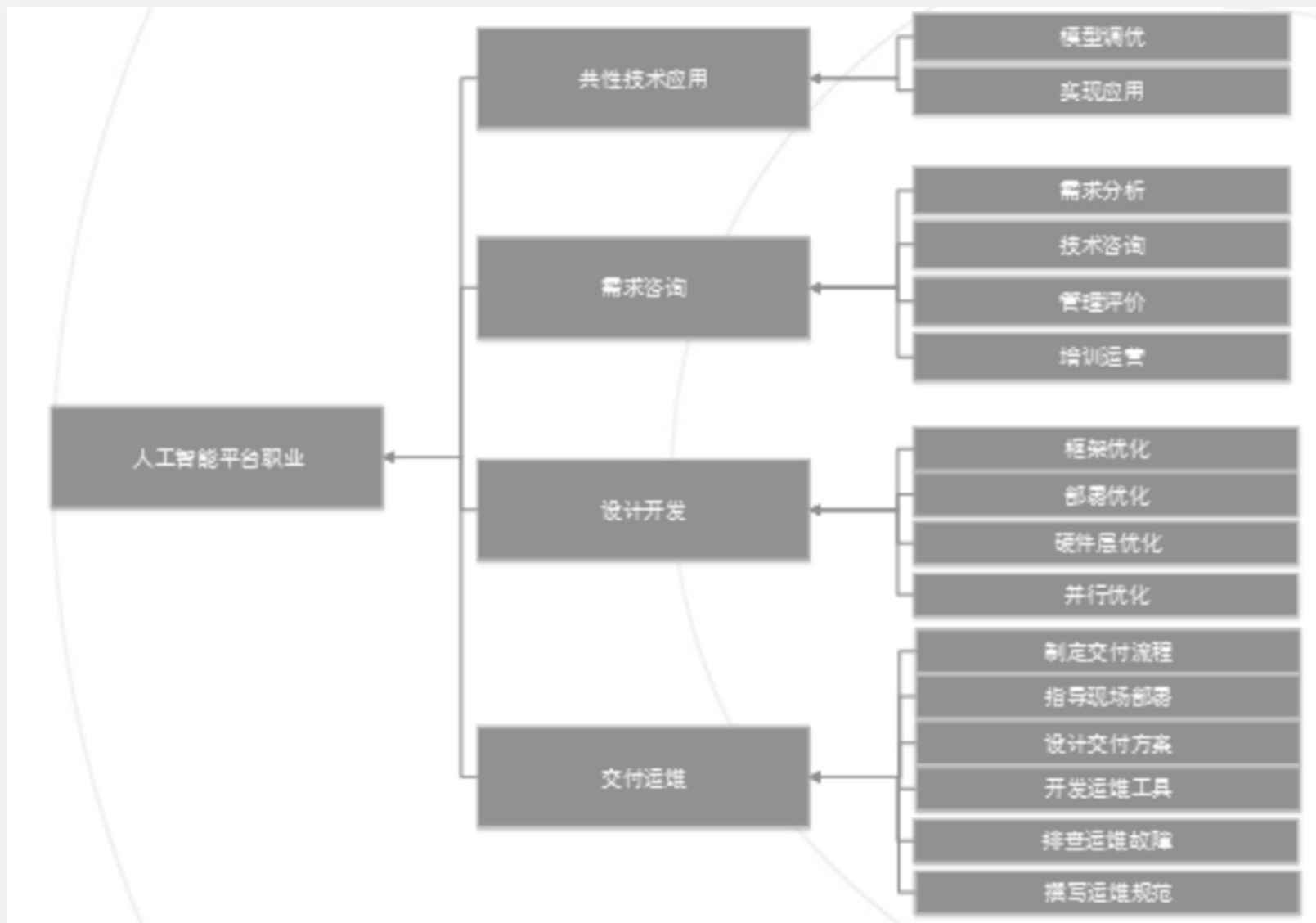
人工智能工程技术人员在企业中的最终角色是CTO（首席技术官），其职业通道大致可分为**初级工程技术人员、中级工程技术人员、高级工程技术人员**。

- 初级工程技术人员在企业扮演的角色为：负责功能的实现方案设计、编码实现、疑难BUG分析诊断、攻关解决。
- 中级工程技术人员在企业扮演的角色为：完善初级工程技术人员的需求设计，掌握多种应用场景下的流程和实现细节，管理平台产品的升级、性能优化、版本迭代。
- 高级工程技术人员在企业扮演的角色为：组建平台研发部，搭建公共技术平台，方便上面各条产品线开发；通过技术平台、通过高一层的职权，管理和协调各个产品线组。现在每个产品线都应该有合格的研发经理和高级程序员了。

第二节 人工智能应用典型案例

人工智能工程技术职业发展

人工智能平台产品实现所需的人才涵盖了**共性技术应用、需求咨询、设计开发、交付运维**等领域，具体需要的人才类型可分为以下几种类型，如图所示。



第二节 人工智能应用典型案例

人工智能工程技术职业发展

单位：美团

岗位：大模型-算法-实习生。

岗位职责

- 1、参与大模型算法研究相关工作，包括算法设计、优化和实现；
- 2、通过微调/强化学习、预训练等技术方法，提升模型在各类场景下能力；
- 3、跟踪和研究业界SOTA算法和技术，完成算法的调研、选型与调优；

岗位基本要求

- 1、扎实的算法基础，熟悉LLM、NLP相关算法和模型；
- 2、有Tensorflow、pytorch等深度学习框架与自然语言处理结合实际项目经验者优先
- 3、有语义理解、对话系统、问答系统、机器翻译、知识图谱等相关项目经验者优先。
- 4、在ACL、EMNLP、NAACL、SIGIR等自然语言处理会议或期刊中有论文发表者优先。
- 5、每周至少全职工作四天，保证实习至少4个月。

第二节 人工智能应用典型案例

人工智能工程技术职业发展

单位：蚂蚁

岗位：自然语言处理-算法工程师。

职位要求

必须具备的：

本科及以上学历，硕士博士优先，计算机、数学、信息管理等相关专业；

具备极佳的工程实现能力，精通C/C++、Java、Python、Perl等至少一门语言；

对目前主流的深度学习平台：tensorflow、pytorch、mxnet等，至少对其中一个有上手经验；

熟悉深度学习以及常见机器学习算法的原理与算法，能熟练运用聚类、分类、回归、排序等模型解决有挑战性的问题，有大数据处理的实战经验；

有强烈求知欲，对人工智能领域相关技术有热情；

具有良好的数学基础，良好的英语阅读能力；

学习能力强，对新事物保有好奇心，并能快速适应新环境；

良好的沟通能力和团队协作能力；能与他人合作，共同完成目标；

对所在领域有热情，相信方法总比困难多，善于独立思考并反思总结。

可以加分的：

精通自然语言处理领域的1到2项底层技术，有实际成果并发表在自然语言处理国际顶级会议、期刊；

作为重要角色参与领域内有含金量的比赛并取得成绩（比如ACM）。

第二节 人工智能应用典型案例

人工智能工程技术职业发展

单位：中国电信大数据和AI中心

岗位：机器学习平台开发工程师。

岗位职责：

- 1.负责AI平台整体产品落地能力应用结合工作；
- 2.对标AI应用能力需求，制定平台建设规范，并与产品、研发协作，推动落地；
- 3.参与平台产品规划、反馈分析等工作，制定量化指标验证平台；
- 4.主动分析平台产品底层训练、发布、核心编排、异构算力调度、分布式计算、分布式存储等方向体验问题，并制定解决方案；
- 5.在技术方向上不断提升探索引导 AI 平台发展，协同各角色共同推进 AI能力平台产品的领先地位。

任职要求：

- 1.全日制本科及以上学历，计算机、自动化、模式识别等相关专业；身体健康，年龄原则上在35周岁及以下，条件优秀的可适当放宽；
- 2.3年及以上AI平台相关研发及使用经验，有成熟的案例；
- 3.熟悉AI、Docker、Kubernetes等主流技术，具备相关AI能力平台的开发和使用经验；构建过中等规模以上真实业务场景下的机器学习实际应用项目，并承担核心职能；
- 4.熟悉Tensorflow等至少一种机器学习框架；
- 5.有强烈的工作责任心，良好的学习能力、沟通能力和自驱力；

第二节 人工智能应用典型案例

人工智能工程技术职业发展

单位：上海市大数据中心

岗位：项目管理与应用推进。

序号	岗位名称	岗位类别	岗位等级	岗位职责	招聘人数	招聘对象	最低工作年限	政治面貌	年龄上限	学历要求	学位要求	专业要求	其它条件	备注
11	项目管理与应用推进 (城市建设)	专技岗位	初级或中级	负责信息系统建设规划、总体架构设计和建设管理等工作。	2	不限	不限	不限	35	本科及以上	学士及以上	理工类、管理类相关专业	有架构设计、移动端设计、软件开发、项目管理等相关经验；掌握项目管理、软件工程、大数据、云计算、人工智能、区块链等相关技术知识；有较强沟通协调、逻辑思维能力；有互联网开拓精神和团队合作、拼搏精神；有较强的文字归纳和表达能力。	研究生优先
12	应用推进 (综合领域)	专技岗位	初级或中级	负责系统建设和应用推进：负责应用系统可行性调研、预算编制、方案设计、用户交流、开发测试、试运行、用户培训、上线推广、系统验收等项目建设全过程技术管理和跟踪协调，做好项目文档工作。	6	不限	3年	不限	35	本科及以上	学士及以上	计算机软件相关专业	1、熟悉项目管理、软件工程、大数据、人工智能、区块链等相关知识； 2、具备构架设计、软件开发、项目管理、移动端设计等经验； 3、具备较强的独立思考、逻辑思维能力，有良好的团队协作意识和沟通协调能力； 4、有大型软件开发经验优先。	

第二节 人工智能应用典型案例

自然语言处理与智能对话

智能客服机器人是基于自然语言处理（*NLP*）和智能对话技术构建的，旨在为用户提供高效、个性化的服务和支持。

- 一家电商平台可以利用智能客服机器人来处理用户的咨询和问题。当用户在网站上或移动应用上遇到问题时，他们可以通过聊天界面与智能客服机器人进行交流。机器人可以通过*NLP*技术理解用户的提问，并根据用户的需求提供相应的解答和建议。例如，用户可能询问某个商品的价格、尺寸、颜色等信息，智能客服机器人可以根据用户的问题从数据库中检索相关信息，并以自然的语言进行回复。
- 除了回答常见问题外，智能客服机器人还可以处理复杂的交互场景，例如处理用户的投诉、退款申请、订单跟踪等。通过不断学习和优化，智能客服机器人可以逐渐提高对用户问题的准确理解和解决能力，从而提升用户体验和服务质量。
- 这样的智能客服机器人应用案例，不仅可以提高企业的客户服务效率，降低人力成本，还能够为用户提供**24*7**的在线支持，提升用户满意度和忠诚度。

第二节 人工智能应用典型案例

自然语言处理与智能对话

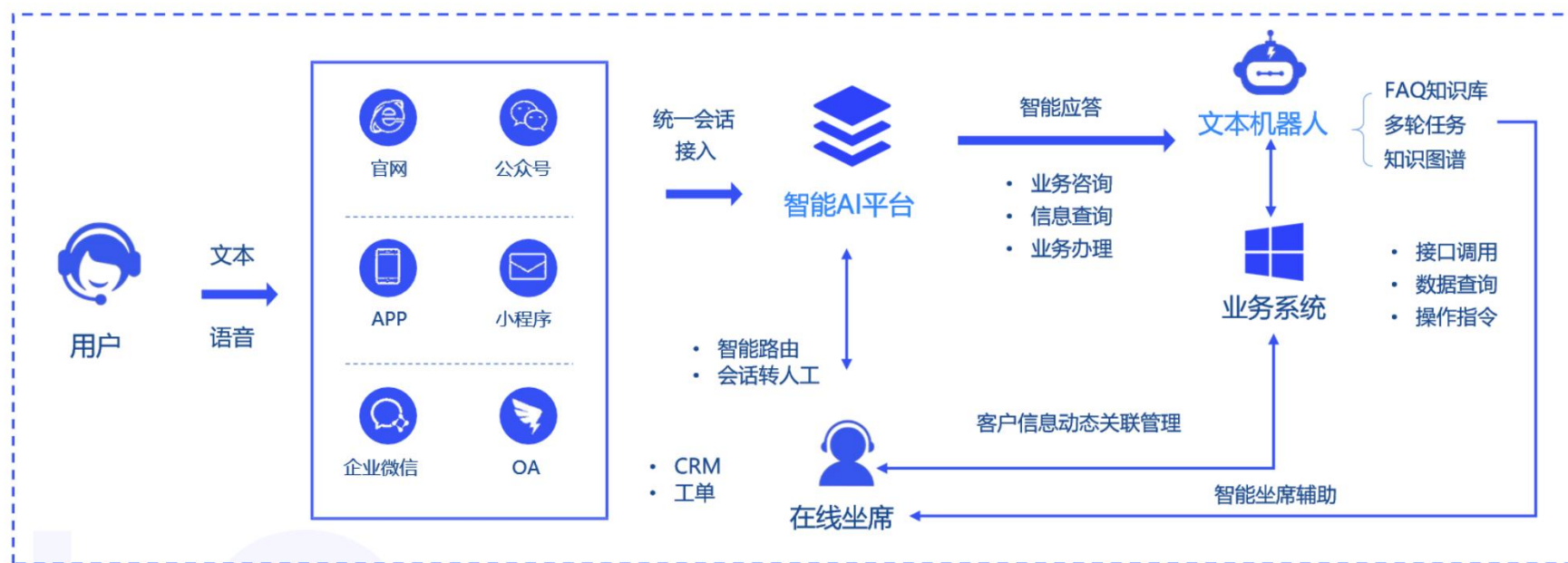
来源：金融智能化转型 - 平安银行AI中台架构搭建实战



智能客服系统-业务运行流程

机器人智能应答，自助完成业务咨询、查询和办理

强大的自然语言问答能力，有效拦截 80%以上简单、重复性问题



全渠道一键接入

智能机器人问答

智能知识库

人机协作服务

跨系统数据交互

第二节 人工智能应用典型案例

图像识别与计算机视觉

智能安防监控系统利用图像识别和计算机视觉技术，对监控摄像头拍摄到的视频图像进行实时分析和处理，以实现对环境中人、车辆、物体等目标的识别和监测。

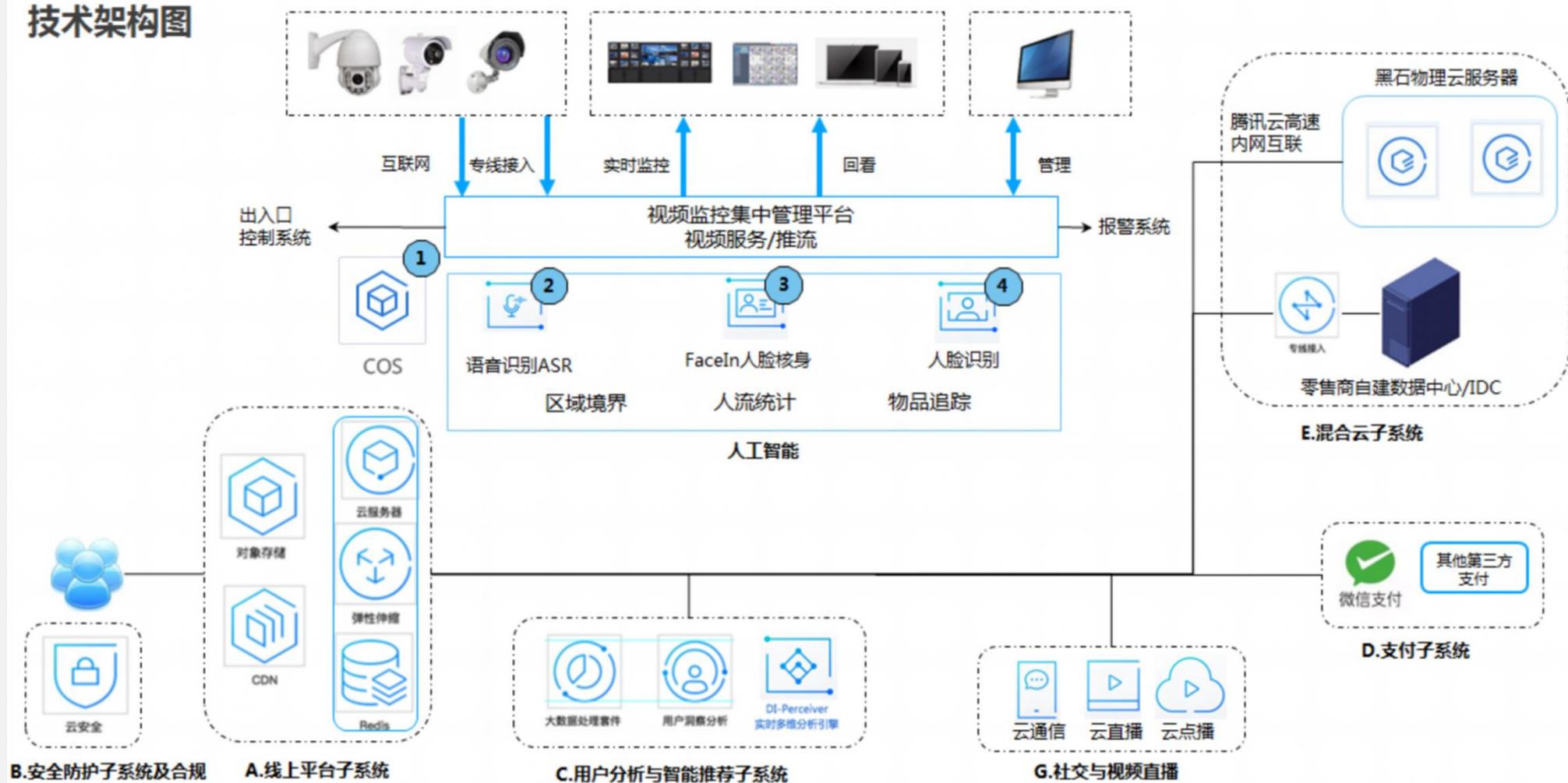
- 人脸识别：识别出场景中的人脸，并进行身份验证或者比对，用于识别进出人员、追踪嫌疑人等。
- 车辆识别：识别出场景中的车辆，并进行车牌识别，用于车辆管理、交通违章监测等。
- 物体检测与跟踪：检测出场景中的特定物体，如箱包、武器等，并进行实时跟踪，用于异常物品检测、失物拾取等。
- 行为分析：分析监控画面中的人员行为，如打架、拥挤、倒地等，用于及时发现并处理突发事件。
- 场景分析与区域监控：对监控场景进行智能分析，划分监控区域，设定安全警戒线，实现对重要区域的实时监控。

第二节 人工智能应用典型案例

图像识别与计算机视觉

来源：电商智能门店管理与安防监控-腾讯云

技术架构图



第二节 人工智能应用典型案例

智能推荐系统与决策支持

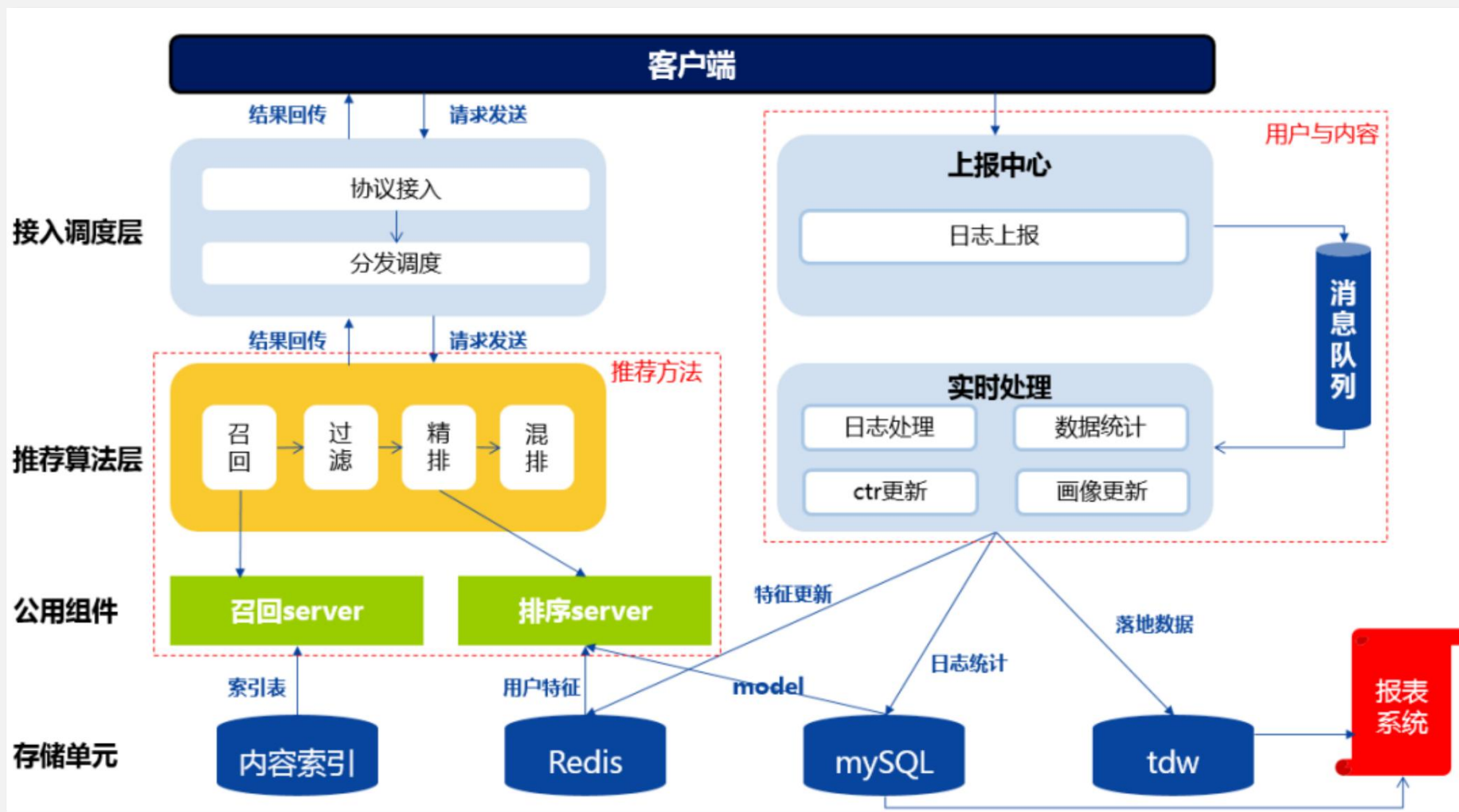
智能推荐系统通过分析用户的历史行为、偏好和兴趣，以及物品的属性和特征，利用各种推荐算法生成个性化的推荐结果，向用户推荐他们可能感兴趣的物品或内容。这种推荐可以涵盖各种领域，如电商平台的商品推荐、社交媒体的好友推荐、音乐视频的推荐等。

- 个性化决策支持：结合用户的个性化偏好和需求，为用户提供个性化的决策支持，使决策更加贴近用户的实际情况和需求。
- 实时推荐与决策：结合实时数据分析和实时决策模型，实现实时的推荐和决策支持，帮助用户在动态变化的环境中做出即时的决策。
- 增强用户体验：通过更准确的推荐和更智能的决策支持，提高用户体验和满意度，增强用户对系统的信任和忠诚度。
- 优化业务效率：基于数据驱动的推荐和决策支持，可以帮助企业优化业务流程和资源配置，提高业务效率和竞争力。

第二节 人工智能应用典型案例

智能推荐系统与决策支持

来源：从零开始了解推荐系统全貌 - yijiapan, 腾讯WXG数据科学



第二节 人工智能应用典型案例

智能物联网与智能家居

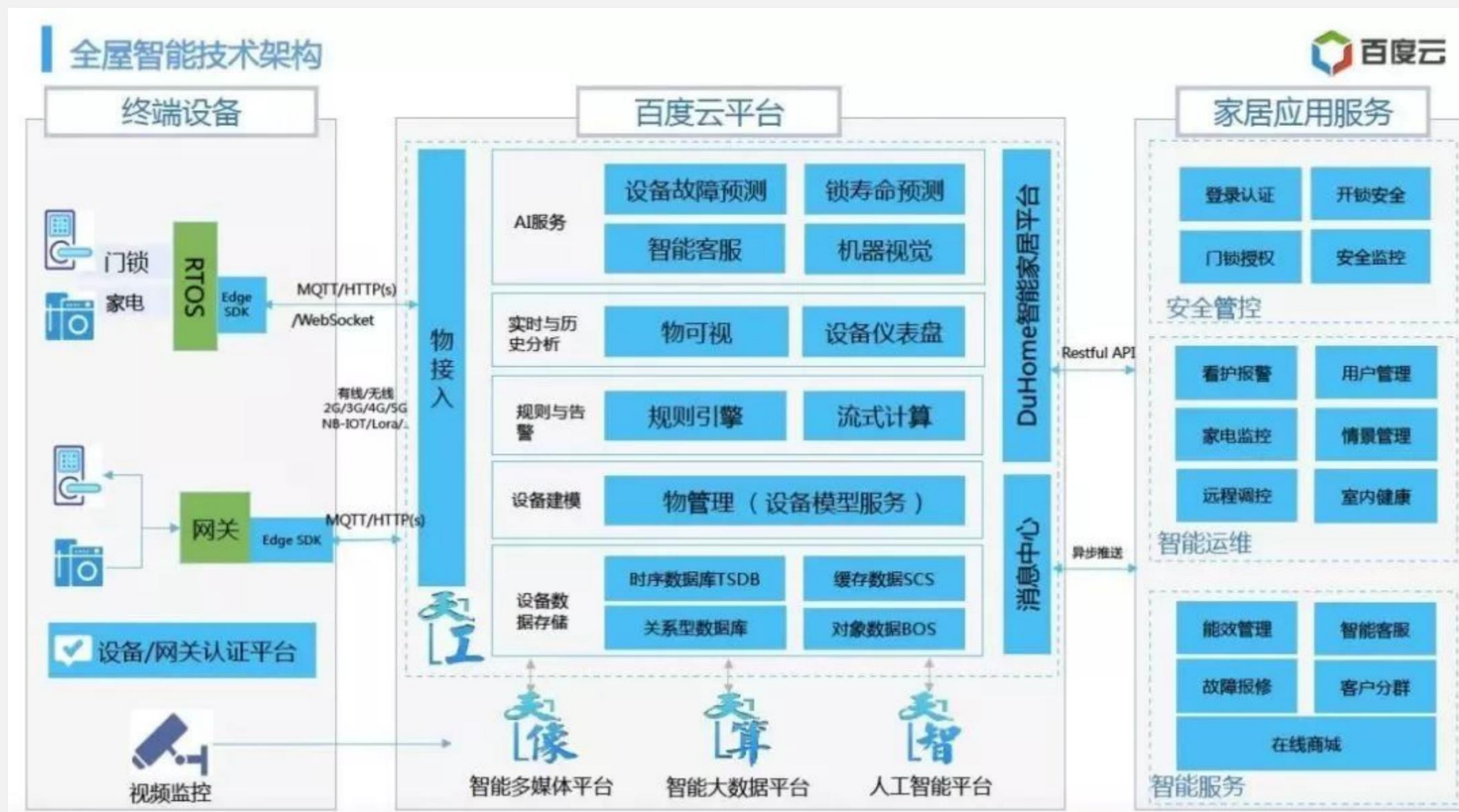
智能环境监测系统利用物联网和人工智能技术，对家庭环境中的各种参数进行实时监测和分析，从而实现智能化的环境管理和控制。

- 温度和湿度监测：通过安装温湿度传感器，实时监测家庭内部的温度和湿度情况，以确保舒适的居住环境。
- 空气质量监测：配备空气质量传感器，监测室内空气中的污染物浓度，如 $PM_{2.5}$ 、 CO_2 等，帮助用户及时采取措施改善室内空气质量。
- 光照和声音监测：使用光照传感器和声音传感器，监测室内光照强度和環境噪音水平，以调节室内光线和保持安静的生活环境。
- 能耗监测：连接智能电表或插座，监测家庭电器的能耗情况，帮助用户管理和优化能源使用。
- 智能控制与调节：结合人工智能算法，系统可以根据监测数据自动调节家庭设备的运行状态，如智能空调、智能照明等，实现能耗节约和舒适性的提升。
- 远程监控与控制：用户可以通过手机App或云平台远程监控和控制家庭环境，无论身在何处都能够实现对家庭的智能管理。

第二节 人工智能应用典型案例

智能物联网与智能家居

来源：《度家DuHome：让智能家居的梦想照进现实》



第二节 人工智能应用典型案例

主流人工智能平台示例

腾讯AI开放平台: <https://ai.qq.com/>

人脸检测与分析

五官定位

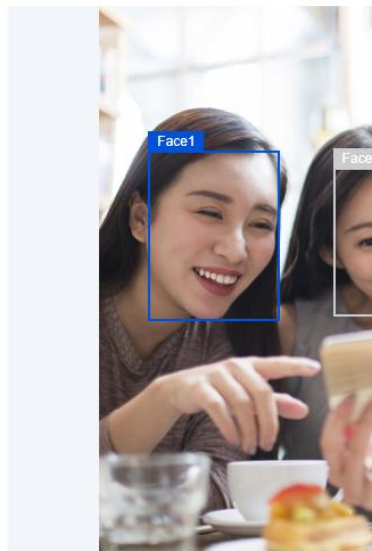
人脸比对

人脸搜索

人脸检测与分析

核心能力

对给定的人脸图片，检测人脸位置、人脸面部属性（包括性别、年龄、表情等）。[了解详情 >](#)



人脸比对

核心能力

对给定两张图片中的人脸进行相似度比对，返回人脸相似度分数，实现1:1比对。[了解详情 >](#)



人脸搜索

参数设置

图片质量控制 不进行检测

识别结果

是同一个人的可能性很高(100分)

Request

Response

第二节 人工智能应用典型案例

主流人工智能平台示例

百度AI开放平台: <https://ai.baidu.com/>



03

人工智能平台开发流程

- 人工智能平台需求分析
- 人工智能平台设计开发
- 人工智能平台测试验证
- 人工智能平台产品交付与运维

第三节 人工智能平台开发流程

人工智能平台需求分析

在人工智能平台的需求分析阶段，首先需要与业务团队和利益相关者进行沟通，了解他们的需求和期望。然后，通过调研和分析，收集并整理各方面的需求，包括功能需求、性能需求、安全需求等。

互联网时代的产品经理构建的是基础设施，在人与人、人与物、人与数据的关系上构建桥梁，实质是优化了信息储存和互通的方式，因此产品经理主要关注入口及流量的走向。人工智能带来的是技术创新驱动下的产业升级，本质上是关注产品本身的价值。与需求相关的职责，是对产品经理最基本的要求，包括需求挖掘、需求管理、需求定义、需求确认等。

第三节 人工智能平台开发流程

人工智能平台需求分析

- 宏观：产品经理要对公司的整体产品架构有清晰认识，评估需求是否符合公司的整体战略规划。
- 微观：产品经理要给出明确的业务背景和业务目标，尽可能对目标进行量化。
 - 安全性：如居家机器人，因为要获得用户的视频、音频和各种家庭内部传感器采集的数据源，因此此类产品需要尽量将数据储存在本地，并实现局域网内部不同物联网设备的本地通信对话机制。
 - 可用性：包括易用性、一致性、观感需求等。其中一致性原则包括且不仅限于三方面：设计目标一致性、外观元素一致性、交互行为一致性。
 - 可靠性：常见的可靠性包括出错频率、自我恢复速度；在一些本身对稳定性要求较高的场景中尤其重要，如金融交易、医疗诊断、交通驾驶；
 - 性能：包括响应时间（**2/5/10秒**原则，**2秒**内响应用户体验较好，**5秒**内响应用户体验一般，**10秒**内响应可以接受，超过**10秒**无法接受）、吞吐量、并发用户数（指同一时间段内访问系统的用户数量）、资源利用率（要明确上限以应对性能瓶颈的意外来临）。
 - 可支持性：包括可扩展性、可维护性、可安装性（在不同环境下部署安装产品所需要付出的代价）。

第三节 人工智能平台开发流程

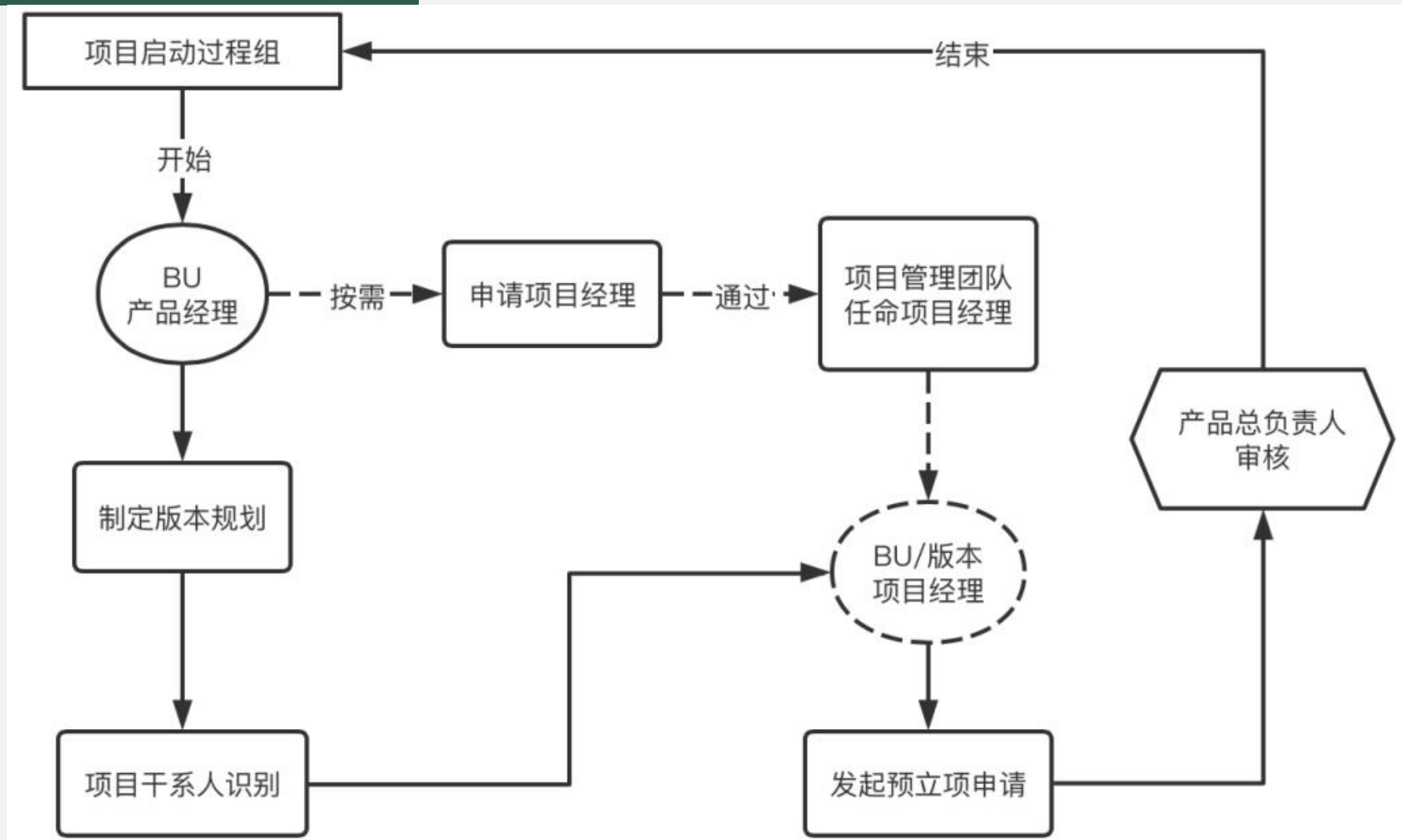
量化需求的方法

产品经理不仅要熟悉产品使用场景，还要了解技术边界，对产品进行量化定义。

- 明确产品的业务需求：包括业务机会、业务目标、成功标准以及产品愿景（包括宏观目标和微观目标）。
- 找准需求的场景：产品经理需要分析每个目标对应的使用场景；如目前的AI是弱人工智能，单个产品只能在某一个特定的具体任务上表现出应用价值，如不拆分，则无法量化。
- 定义场景中的可量化标准：即使在同一领域中，不同场景下对算法的评估标准都完全不同，因此可以看到产品经理需要成为行业专家才不会被淘汰。

第三节 人工智能平台开发流程

需求管理流程图



第三节 人工智能平台开发流程

需求文档撰写

产品需求文档（Product Requirement Document, PRD），主要用于完整描述产品需求，向研发部门明确产品的功能及其性能要求。PRD是项目启动之前，必须要通过评审确定的最重要文档。PRD的主要使用对象有：开发、测试、项目经理、交互设计师、运营及其他业务人员。开发可以根据PRD获知整个产品的逻辑；测试可以根据PRD建用例；项目经理可以根据PRD拆分工作包，并分配开发人员；交互设计师可以通过PRD来设计交互细节。

主要内容包括对项目的介绍，包括项目概述、项目价值、项目背景等；

PRD的基本要求：

完整、准确、清晰、简洁、稳定

第三节 人工智能平台开发流程

需求文档撰写

- **点：** 产品功能列表（版本说明），包括做什么功能、具体功能描述、为什么要做、紧急情况、商业价值、前后台各角色的关系；
- **线：** 产品的整体业务逻辑、如何运转、用户实际要操作的流程、信息架构图；
- **面：** 前后台页面，先画页面流程图让别人有个大致了解，然后逐页说明，先阐述页面提供的功能，然后进行结构说明。对于交互说明，可以把页面拆解成元素/字段（比如如何读取、显示字数和文字的限制、表单默认值、页面无数据等）、规则（哪个角色访问、从什么页面进入、列表排序规则、默认排序、是否有缓存、分页功能、业务层面的规则等）、操作三部分。

第三节 人工智能平台开发流程

写作逻辑—三步分解需求

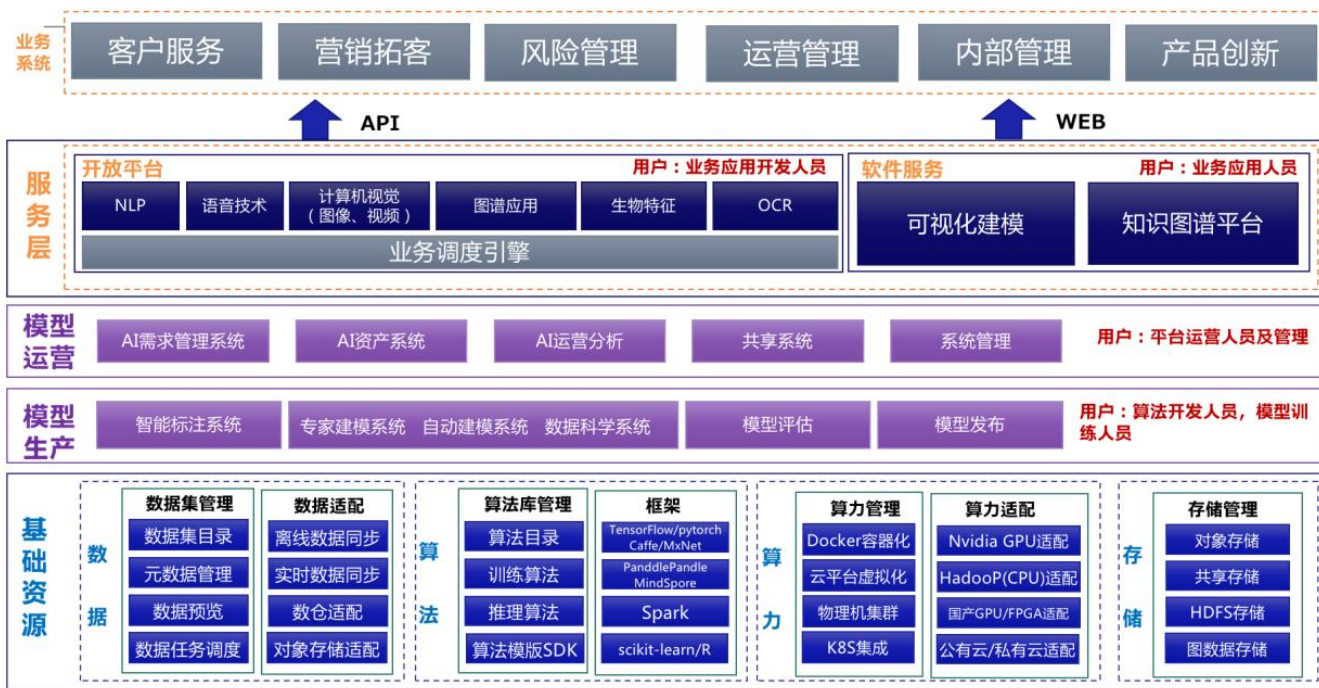
- 理清需求（**user case**）：把想要做的事情和用户可能会需要的功能，以及相关场景罗列出来。
- 整理故事（**user story**）：故事需要核心主题（主线）。讲法有多种，可以绘声绘色的讲（视觉效果）、切入人心的讲（用户心理）、抓住重点的讲（核心流程）等。
- 分解故事（**functional requirement**）：用户需求的整理，先考虑用户需求是什么，再考虑用户可以从产品得到什么（产品的价值），然后考虑如何满足不同用户的使用场景（用户场景，**user case**），最后考虑产品应该做什么（功能需求，用户输入什么，系统输出什么）

第三节 人工智能平台开发流程

人工智能平台架构设计

1. 用户界面（UI）设计
2. 后端服务架构设计
3. 数据架构设计
4. 算法模型架构设计
5. 计算资源管理架构设计
6. 安全架构设计
7. 监控与日志架构设计

AI平台产品架构



人工智能平台架构设计

用户界面（UI）设计

包括用户界面的设计和实现，包括平台的交互界面、可视化界面等。UI设计需要考虑用户友好性、易用性和美观性。确定用户界面的整体布局、交互方式、页面组件等，以及选择合适的前端开发框架和技术（如React、Angular、Vue.js等）。

- 用户友好性：界面设计应当简单直观，易于理解 and 操作。用户应该能够快速找到需要的功能，完成任务。
- 易用性：界面设计应当考虑用户的使用习惯和期望，避免过度复杂的操作流程和不必要的交互步骤。
- 美观性：界面设计应当注重视觉效果和用户体验，采用合适的颜色、布局和图标，使界面看起来美观、舒适、清晰。

人工智能平台架构设计

后端服务架构设计

包括服务器端的架构设计和实现。这包括确定后端服务的组织结构、功能模块划分、数据处理流程等，负责处理用户请求、管理数据、执行算法和模型，并提供对外服务，选择合适的后端开发框架和技术（如Spring Boot、Django、Flask等）。

- 服务拆分： 首先，根据功能和需求，将后端服务拆分成多个独立的模块或微服务。常见的模块包括**用户管理模块**、**数据管理模块**、**算法执行模块**等。这样的设计有助于提高系统的可扩展性和灵活性。
- 服务间通信： 设计服务间的通信机制，包括同步调用和异步消息传递。常用的通信方式包括**RESTful API**、消息队列、**RPC**等。合理的通信机制能够确保服务之间的高效协作和数据传递。

人工智能平台架构设计

数据架构设计

负责管理、存储和处理平台所涉及的数据，包括用户数据、训练数据、模型数据等。

- 数据存储选择： 根据数据需求和业务特点，选择合适的数据存储技术和方案。常见的数据存储包括**关系型数据库**、**NoSQL数据库**、**分布式文件系统**等。针对大规模数据和高并发访问的场景，可能需要选择分布式存储方案。
- 数据模型设计： 设计合适的数据模型，包括数据表结构、数据字段、数据关系等。在设计数据模型时，需要考虑数据的**组织结构**、**存储格式**、**索引策略**等方面，以提高数据的查询效率和存取速度。
- 数据流程设计： 设计数据的流动和处理流程，包括数据的**采集**、**存储**、**处理**、**分析和展示**等环节。这涉及到数据的流向、转换规则、数据管道的搭建等内容。

算法模型架构设计

涉及如何设计和组织算法模型的结构、功能和交互。

- 算法选择： 根据需求分析的结果，选择合适的算法和模型来解决问题。根据不同的应用场景，可能需要选择机器学习算法、深度学习模型、传统的统计方法等。
- 模型结构设计： 设计算法模型的整体结构，包括模型的层次结构、网络拓扑、参数设置等。根据问题的复杂程度和数据的特点，确定模型的复杂度和灵活性。
- 数据预处理： 设计数据预处理流程，包括数据清洗、特征提取、特征选择等环节。数据预处理是模型训练的重要前置步骤，直接影响模型的性能和效果。
- 模型训练与调优： 设计模型的训练和调优策略，包括损失函数的选择、优化算法的调参、模型参数的更新等。通过不断迭代和调整，提高模型的准确率和泛化能力。
- 模型评估与验证： 设计模型评估和验证方法，包括交叉验证、测试集验证、指标评估等。通过评估和验证，检验模型的性能和稳定性，确保模型的有效性和可靠性。
- 模型部署与集成： 设计模型的部署和集成方案，包括模型的打包、接口设计、部署环境的配置等。将训练好的模型部署到平台上，并集成到实际应用中，以实现算法的应用和价值。

资源与安全架构设计

- 资源池设计：设计计算资源的池化结构，包括服务器集群、虚拟化资源、容器资源等。根据需求和预算，确定资源池的规模和配置。
- 资源调度策略：设计资源调度的算法和策略，包括任务调度、计算节点分配、资源优先级等。根据任务的特点和优先级，合理分配和管理计算资源，确保任务的及时执行和高效利用。
- 负载均衡：设计负载均衡的机制和算法，平衡计算资源的负载，防止资源过载或空闲。通过动态调整资源分配，提高资源利用率和系统的稳定性。
- 监控与调优：设计监控和调优的方案，实时监控计算资源的使用情况和性能指标，及时发现和处理资源瓶颈和故障，优化资源配置和调度策略。
- 弹性扩展：设计弹性扩展的机制和策略，根据需求和负载情况，自动调整计算资源的数量和规模，实现平台的动态伸缩和弹性部署。
- 安全管理：设计安全管理的机制和策略，保护计算资源的安全和隐私，防止恶意攻击和非法访问。采取措施保障数据和系统的安全性和稳定性。

第三节 人工智能平台开发流程

人工智能平台技术选型



django



TensorFlow

PYTORCH



技术选型是根据需求和设计目标选择合适的技术框架和工具。这涉及到数据库的选择（如MySQL、MongoDB、Redis等）、编程语言的选择（如Python、Java、Go等）、开发框架的选择（如TensorFlow、PyTorch、Django、Spring等）等。在进行技术选型时，需要综合考虑技术的成熟度、社区支持、性能表现、安全性等因素，以及团队成员的技术能力和熟悉程度。

人工智能平台开发实现

在开发实现阶段，开发团队将根据设计文档和技术选型结果，具体实现人工智能平台的各个功能模块。

- 分工协作：开发团队根据模块设计和任务分配，合理分工，协作完成各个模块的开发工作。团队成员需要相互配合，及时沟通，解决开发过程中遇到的问题。
- 编码实现：开发团队根据需求和设计文档，进行具体的编码工作。他们会使用选择的编程语言和开发框架，按照规范和设计的要求，实现平台的各个功能。
- 遵循编码规范：开发团队需遵循良好的编码规范，包括命名规范、代码风格、注释规范等，保证代码的清晰易读、易于维护。
- 文档和记录：开发团队及时记录开发过程中的关键信息和决策，包括技术方案、问题解决方案、修改记录等，为后续的维护和迭代提供参考。

第三节 人工智能平台开发流程

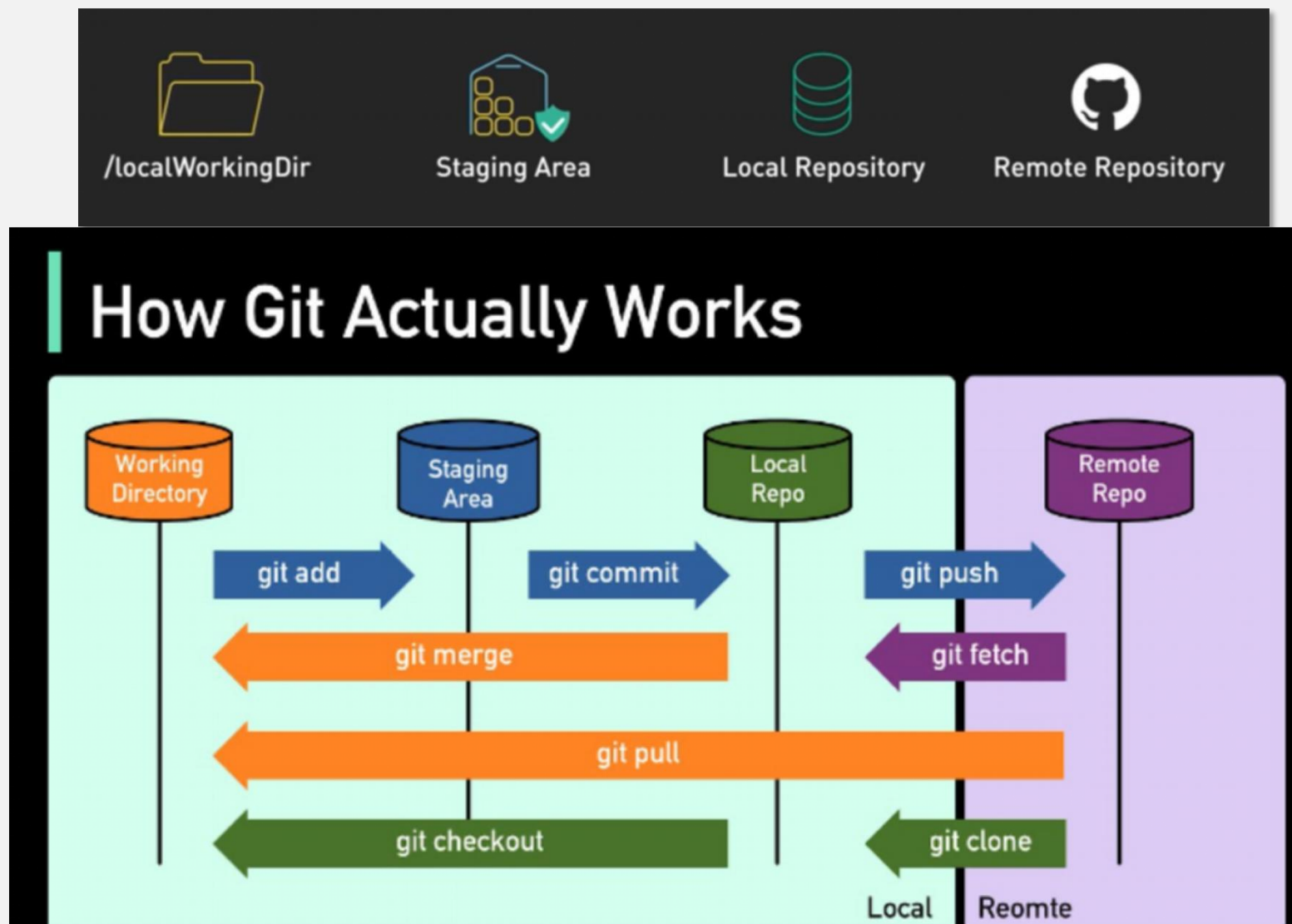
人工智能平台开发实现



- 版本控制: **Git** 可以帮助团队追踪和管理项目的版本历史。每次代码提交都会被记录, 团队可以随时查看项目的变更历史, 并回滚到任意历史版本。
- 团队协作: **Git** 提供了分支 (**branch**) 和合并 (**merge**) 的功能, 使得团队可以并行开发不同的功能模块或解决不同的问题, 而不会相互干扰。团队成员可以在自己的分支上进行开发, 然后通过合并操作将代码整合到主分支中。
- 代码审查: **Git** 的 **Pull Request** 功能允许团队成员向主分支提交自己的代码, 并请求其他成员进行审查。通过代码审查, 团队可以发现潜在的问题、提出改进建议, 并确保代码质量和风格的一致性。
- 远程协作: **Git** 使得团队成员可以在不同的地理位置进行远程协作。通过远程仓库, 团队成员可以共享代码、查看其他人的提交, 并进行实时的协作和交流。
- 备份和恢复: **Git** 提供了强大的备份和恢复功能, 可以保护项目代码免受意外删除或损坏的影响。团队可以定期将代码推送到远程仓库, 确保代码的安全性和可靠性。

第三节 人工智能平台开发流程

人工智能平台开发实现



第三节 人工智能平台开发流程

人工智能平台开发实现

```
okamoto@linux-czfg:...e/TxtFUIO/OneByOne
File Edit View Search Terminal Help
int main(){
    cout << "Input File Name = ";
    char nm[100];
    cin >> nm;
    ifstream fin(nm);
    cout << "Output File Name = ";
    cin >> nm;
    ofstream fout(nm);
    int v;
    fin >> v;
    while (!fin.fail()){
        fout << v << endl;
    }
}
```

```
C:\Windows\system32\cmd.exe
Read from file: Fry:
Read from file: One
Read from file: Jillion
Read from file: dollars.
Read from file: [Everyone
Read from file: gasps.]
Read from file: Auctioneer:
Read from file: Sir,
Read from file: that's
Read from file: not
Read from file: a
Read from file: number.
Read from file: 数据读取, 测试。
/n Read from file: Fry: One Jillion dollars.
Read from file: [Everyone gasps.]
Read from file: Auctioneer: Sir, that's not a number.
Read from file: 数据读取, 测试。
/n Read from file: Fry: One Jillion dollars.
Read from file: [Everyone gasps.]
Read from file: Auctioneer: Sir, that's not a number.
Read from file: 数据读取, 测试。
/n Error opening dataFUNNY.txt for input
请按任意键继续. . .
```

```
home.blade.php - blog
home.blade.php x _colors.scss
@extended('layout.app')
@section('content')
<section class="website-content">
<div id="sub-header">
<div class="height-20p"></div>
<div class="container">
<section id="feature-post" class="py-5">
<div class="row align-items-center">
<div class="col-lg-7">
<div>
<h1 class="text-lg font-weight-bold text-gray-800">
Having a
Very Good Experience
```

```
horoscope-bot-tutorial main
horoscope-bot-tutorial > main.py
main.py x requirements.txt
34 @bot.message_handler(commands=['horoscope'])
35 def sign_handler(message): message: <telebot.types.Message object at 0x1066b4af0>
36     user_input = f"What's your zodiac sign?\nChoose one: {STAR_SIGNS}." user_input: "What's your zq
37     sent_msg = bot.send_message(message.chat.id, user_input, parse_mode="Markdown")
38     bot.register_next_step_handler(sent_msg, day_handler)
39
Debug main
Threads & Variables Console
WorkerThread2 Evaluate expression (⇧) or add a watch (⌘⇧)
sign_handler, main.py:37
_run_middleware_and_h
run, util.py:91
_bootstrap_inner, threadi
_bootstrap, threading.py;
Switch frames from anywh... X
message = (Message) <telebot.types.Message object at 0x1066b4af0>
animation = (NoneType) None
audio = (NoneType) None
author_signature = (NoneType) None
caption = (NoneType) None
caption_entities = (NoneType) None
channel_chat_created = (NoneType) None
LF UTF-8 4 spaces Python 3.9 (horoscope-bot-tu 2,580 x 1,440
```

VS Code, PyCharm, IntelliJ IDEA...

第三节 人工智能平台开发流程

人工智能平台开发实现



Java开发手册（华山版）.pdf

一、编程规约

(一) 命名风格

1. **【强制】** 代码中的命名均不能以下划线或美元符号开始，也不能以下划线或美元符号结束。
反例：`_name` / `__name` / `$name` / `name_` / `name$` / `name__`
2. **【强制】** 代码中的命名严禁使用拼音与英文混合的方式，更不允许直接使用中文的方式。
说明：正确的英文拼写和语法可以让阅读者易于理解，避免歧义。注意，纯拼音命名方式更要避免采用。
正例：`renminbi` / `alibaba` / `taobao` / `youku` / `hangzhou` 等国际通用的名称，可视同英文。
反例：`DaZhePromotion` [打折] / `getPingfenByName()` [评分] / `int 某变量 = 3`
3. **【强制】** 类名使用 UpperCamelCase 风格，但以下情形例外：`DO` / `BO` / `DTO` / `VO` / `AO` / `PO` / `UID` 等。
正例：`JavaServerlessPlatform` / `UserDO` / `XmlService` / `TcpUdpDeal` / `TaPromotion`
反例：`javaserverlessplatform` / `UserDo` / `XMLService` / `TCPUDPDeal` / `TAPromotion`
4. **【强制】** 方法名、参数名、成员变量、局部变量都统一使用 lowerCamelCase 风格，必须遵从驼峰形式。
正例：`localValue` / `getHttpMessage()` / `inputUserId`
5. **【强制】** 常量命名全部大写，单词间用下划线隔开，力求语义表达完整清楚，不要嫌名字长。
正例：`MAX_STOCK_COUNT` / `CACHE_EXPIRED_TIME`
反例：`MAX_COUNT` / `EXPIRED_TIME`



人工智能平台测试验证

测试验证阶段是在开发实现完成后进行的必要环节，旨在验证平台的功能完整性、性能稳定性和用户体验。在这个阶段，测试团队会对平台进行全面的测试，并确保它符合设计要求和用户期望。

- 功能测试：测试团队会对平台的各个功能模块进行功能测试，确保每个功能都能正常工作，并且符合需求规格和设计文档的要求。这包括对用户界面、后端服务、数据处理等方面的功能进行验证。
- 性能测试：进行性能测试以评估平台的性能表现，包括响应速度、吞吐量、并发性能等方面。性能测试可以帮助确定平台在各种负载条件下的性能瓶颈，并进行优化和调整。
- 稳定性测试：进行稳定性测试以评估平台在长时间运行和高负载下的稳定性。通过模拟各种异常情况和异常输入，测试团队可以发现潜在的稳定性问题，并进行修复和优化。

第三节 人工智能平台开发流程

人工智能平台测试验证

- 兼容性测试： 进行兼容性测试以确保平台能够在不同的操作系统、浏览器和设备上正常运行，并与其他相关软件和硬件兼容。这包括对不同操作系统、浏览器和设备的兼容性进行测试验证。
- 安全性测试： 进行安全性测试以评估平台的安全性和防护能力，包括对潜在的安全漏洞和攻击进行检测和验证。测试团队会尝试各种安全攻击和渗透测试，确保平台的安全性。
- 用户体验测试： 进行用户体验测试以评估平台的易用性和用户友好性。测试团队会从用户的角度出发，测试平台的界面设计、交互流程、操作逻辑等方面，提出改进建议和优化建议。
- 回归测试： 在每次功能修改或优化后，进行回归测试以确保修改不会对原有功能造成影响。这可以帮助保证平台的稳定性和功能完整性。

第三节 人工智能平台开发流程

人工智能平台测试验证

对于人工智能元素的平台进行测试验证，除了上述常规测试类型外，还需要特别关注以下几个方面：

- **模型效果验证：** 对人工智能模型进行效果验证，包括**准确率、召回率、精确度**等指标的评估。通过对模型在不同数据集上的测试，验证其在真实场景中的性能表现。
- **数据质量测试：** 对输入数据的质量进行测试，包括数据的**完整性、准确性、一致性和时效性**等方面。确保输入数据符合模型的要求，以提高模型的准确性和稳定性。
- **数据隐私和安全性测试：** 测试人工智能模型在处理用户数据时的隐私保护和安全性措施，包括**数据加密、权限控制、数据脱敏**等方面。确保平台在数据处理过程中符合相关法律法规和隐私政策。
- **AI算法鲁棒性测试：** 对人工智能算法的鲁棒性进行测试，包括对抗攻击、噪声干扰、数据偏差等方面的验证。确保算法在面对各种异常情况时能够保持稳定性和可靠性。
- **自动化测试和持续集成：** 针对人工智能模型的测试，可以借助自动化测试工具和持续集成技术进行测试自动化和持续集成。通过自动化测试流程，可以提高测试效率和覆盖率，确保平台的稳定性和可靠性。

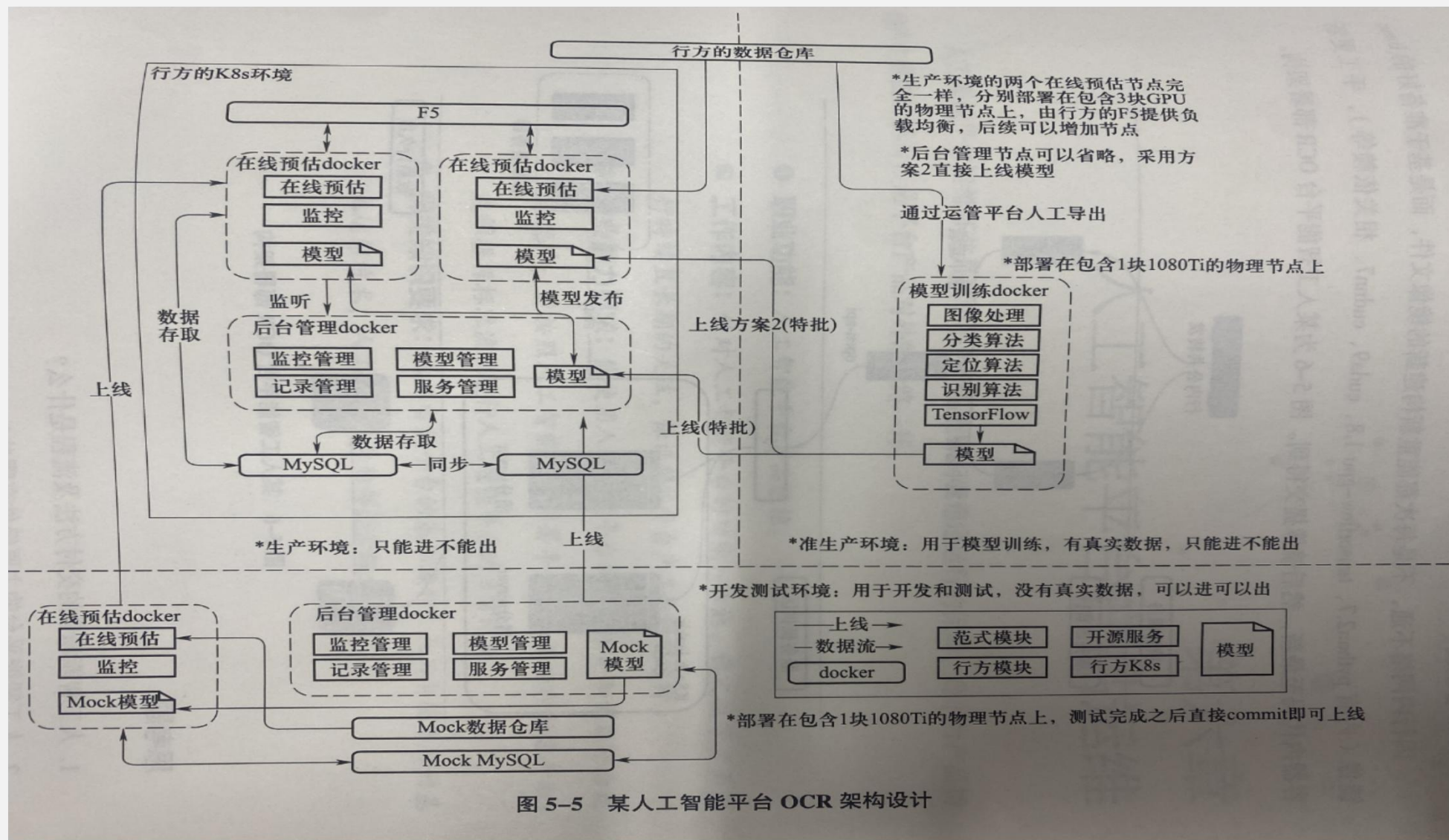
人工智能平台产品交付与运维是确保平台顺利部署、稳定运行并持续提供价值的关键环节。

- 部署和配置：将开发完成的人工智能平台部署到生产环境，并进行必要的配置和参数调整。这包括选择合适的硬件设备、网络环境和操作系统，并进行相应的安装、配置和初始化。
- 数据迁移和转换：将原始数据迁移到生产环境，并进行数据格式转换、清洗和预处理等操作。确保数据在平台中的可用性和准确性。
- 监控和运维工具配置：配置监控和运维工具，对平台的各个组件和服务进行监控和管理。包括监控系统性能、资源利用率、服务运行状态等指标，并设置相应的告警和自动化处理机制。
- 灰度发布和版本管理：使用灰度发布策略，逐步将新版本的平台功能发布到生产环境，并进行版本管理和回滚操作。确保平台在升级和更新过程中不会影响用户体验和业务正常运行。
- 故障排除和问题处理：及时响应和处理平台中出现的故障和问题，包括软件故障、硬件故障、网络故障等。通过日志分析、异常检测和自动化处理等手段，快速定位和解决问题，确保平台的稳定性和可靠性。

第三节 人工智能平台开发流程

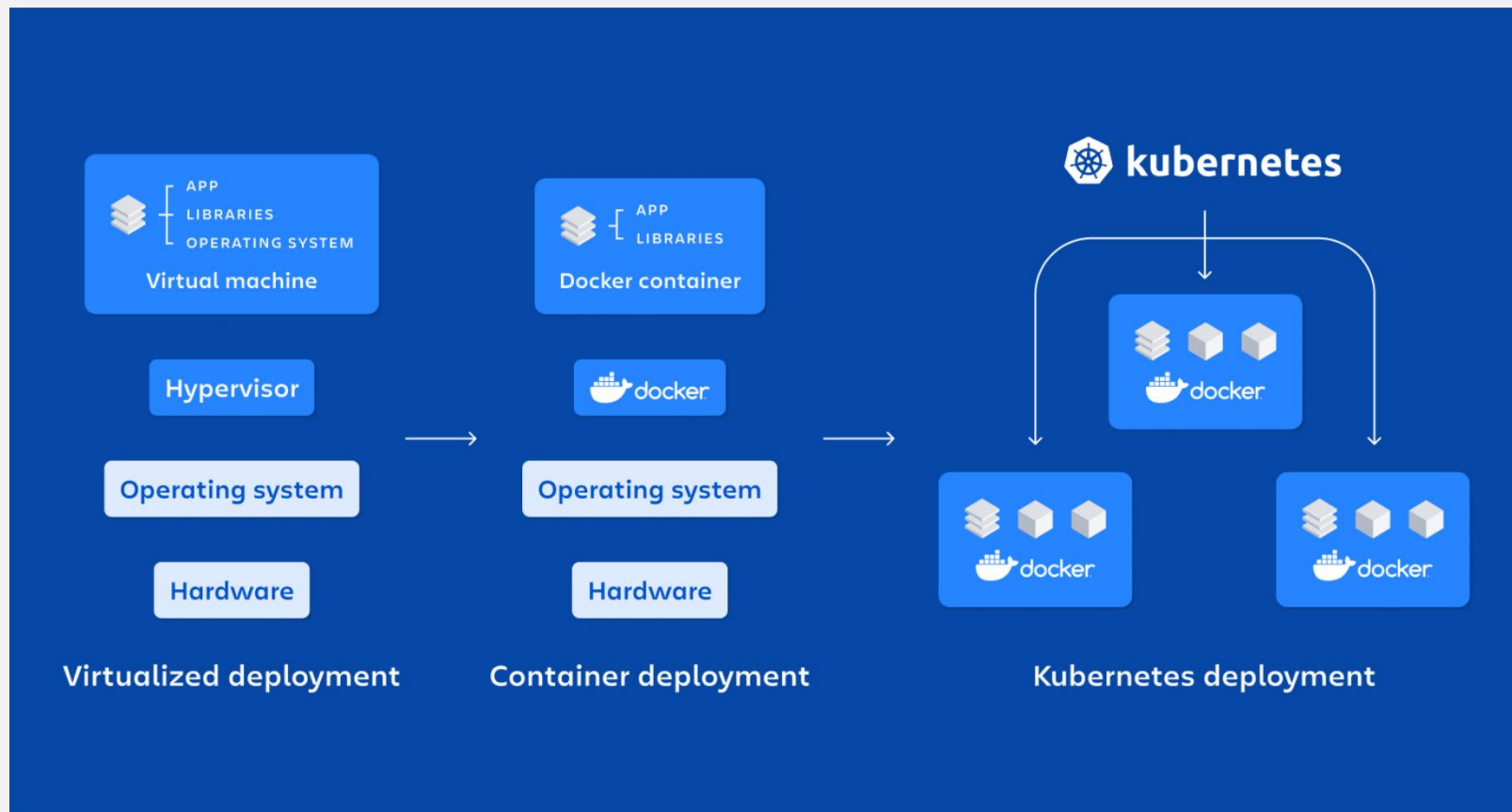
人工智能平台交付与运维

某人工智能平台光学文字识别场景交付实例



第三节 人工智能平台开发流程

人工智能平台交付与运维

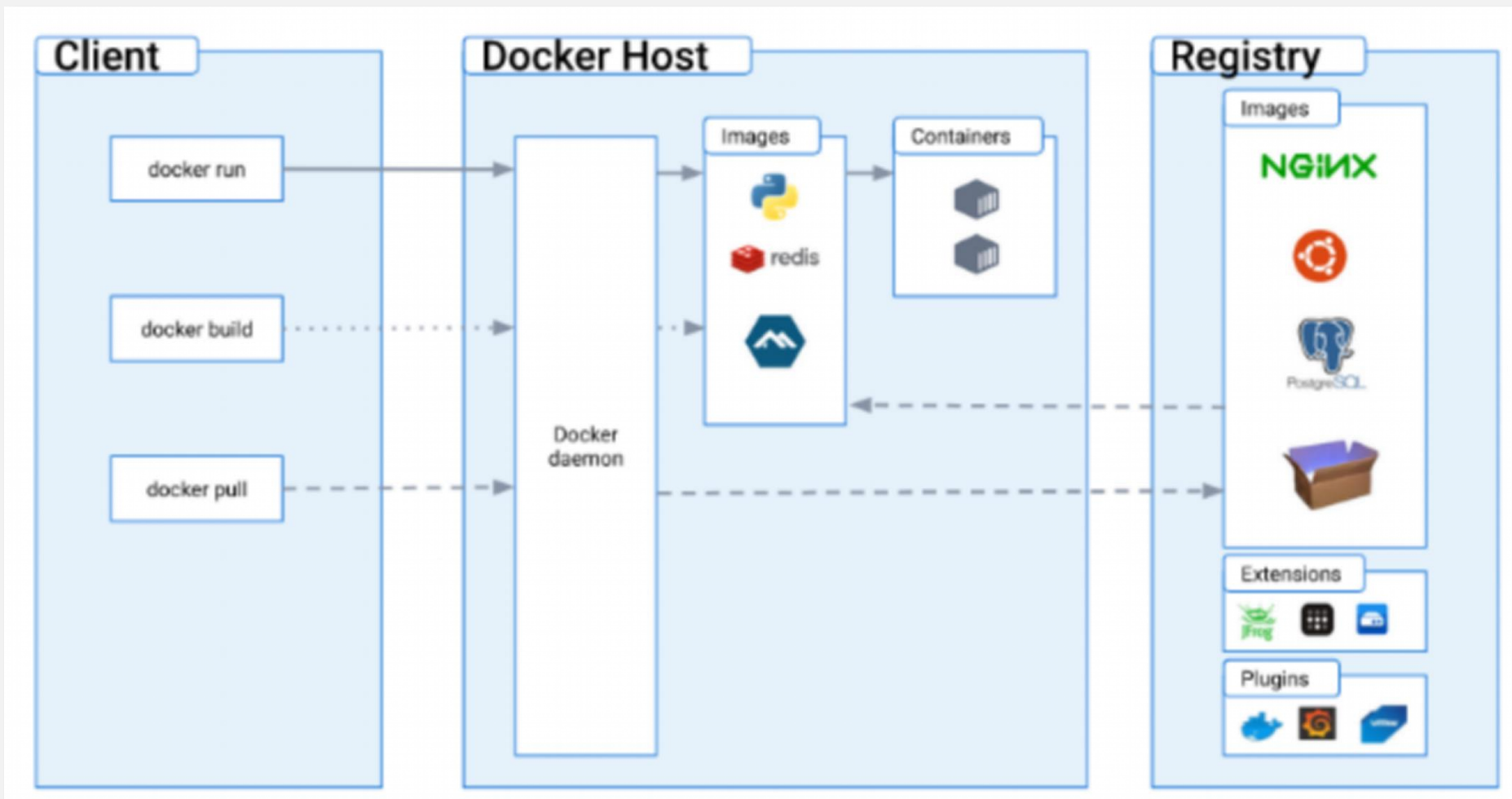


使用容器化技术可以将应用程序及其依赖项打包为一个独立的容器，使得应用程序在不同环境中具有一致的运行表现。**Docker** 是一个流行的容器化平台，而 **Kubernetes** 则是用于容器编排和管理的开源系统，可以实现自动化部署、扩展和管理。

第三节 人工智能平台开发流程

人工智能平台交付与运维

Docker运行机制



第三节 人工智能平台开发流程

人工智能平台交付与运维

Docker常用命令

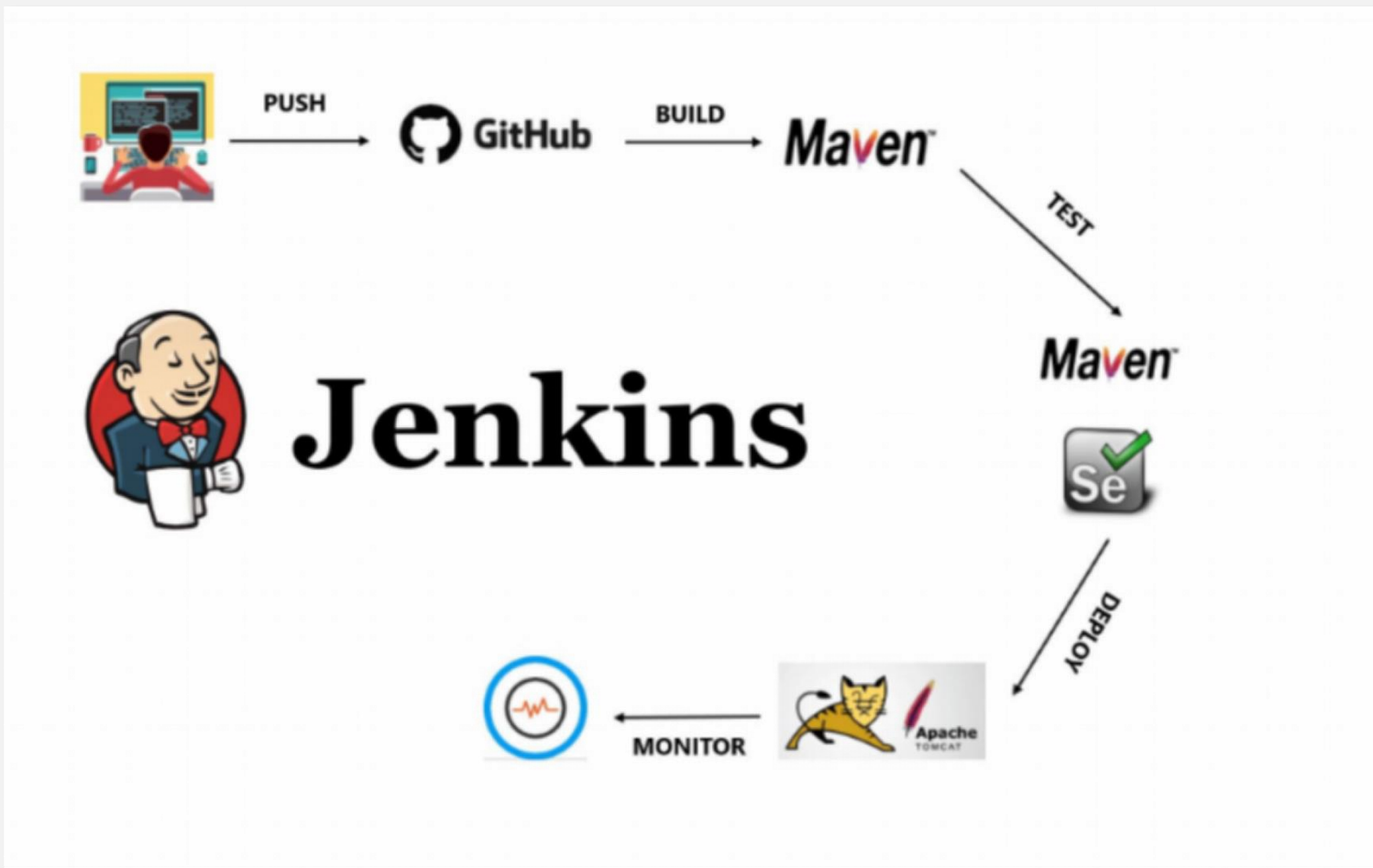


Cheatsheet for Docker CLI

Run a new Container	Manage Containers	Manage Images	Info & Stats
Start a new Container from an Image docker run IMAGE docker run nginx ...and assign it a name docker run --name CONTAINER IMAGE docker run --name web nginx ...and map a port docker run -p HOSTPORT:CONTAINERPORT IMAGE docker run -p 8080:80 nginx ...and map all ports docker run -P IMAGE docker run -P nginx ...and start container in background docker run -d IMAGE docker run -d nginx ...and assign it a hostname docker run --hostname HOSTNAME IMAGE docker run --hostname srv nginx ...and add a dns entry docker run --add-host HOSTNAME:IP IMAGE ...and map a local directory into the container docker run -v HOSTDIR:TARGETDIR IMAGE docker run -v ~/.usr/share/nginx/html nginx ...but change the entrypoint docker run -it --entrypoint EXECUTABLE IMAGE docker run -it --entrypoint bash nginx	Show a list of running containers docker ps Show a list of all containers docker ps -a Delete a container docker rm CONTAINER docker rm web Delete a running container docker rm -f CONTAINER docker rm -f web Delete stopped containers docker container prune Stop a running container docker stop CONTAINER docker stop web Start a stopped container docker start CONTAINER docker start web Copy a file from a container to the host docker cp CONTAINER:SOURCE TARGET docker cp web:/index.html index.html Copy a file from the host to a container docker cp TARGET CONTAINER:SOURCE docker cp index.html web:/index.html Start a shell inside a running container docker exec -it CONTAINER EXECUTABLE docker exec -it web bash Rename a container docker rename OLD_NAME NEW_NAME docker rename 096 web Create an image out of container docker commit CONTAINER docker commit web	Download an image docker pull IMAGE[:TAG] docker pull nginx Upload an image to a repository docker push IMAGE docker push myimage:1.0 Delete an image docker rmi IMAGE Show a list of all Images docker images Delete dangling images docker image prune Delete all unused images docker image prune -a Build an image from a Dockerfile docker build DIRECTORY docker build . Tag an image docker tag IMAGE NEWIMAGE docker tag ubuntu ubuntu:18.04 Build and tag an image from a Dockerfile docker build -t IMAGE DIRECTORY docker build -t myimage . Save an image to .tar file docker save IMAGE > FILE docker save nginx > nginx.tar Load an image from a .tar file docker load -i TARFILE docker load -i nginx.tar	Show the logs of a container docker logs CONTAINER docker logs web Show stats of running containers docker stats Show processes of container docker top CONTAINER docker top web Show installed docker version docker version Get detailed info about an object docker inspect NAME docker inspect nginx Show all modified files in container docker diff CONTAINER docker diff web Show mapped ports of a container docker port CONTAINER docker port web

第三节 人工智能平台开发流程

人工智能平台交付与运维



Jenkins 是一个流行的开源持续集成/持续部署（CI/CD）工具，广泛应用于软件开发项目中，用于自动化构建、测试和部署应用程序。可以通过轮询或监听版本控制系统（如 **Git**、**SVN**）的代码仓库，当检测到代码提交或分支变更时，触发构建任务。

第三节 人工智能平台开发流程

人工智能平台交付与运维

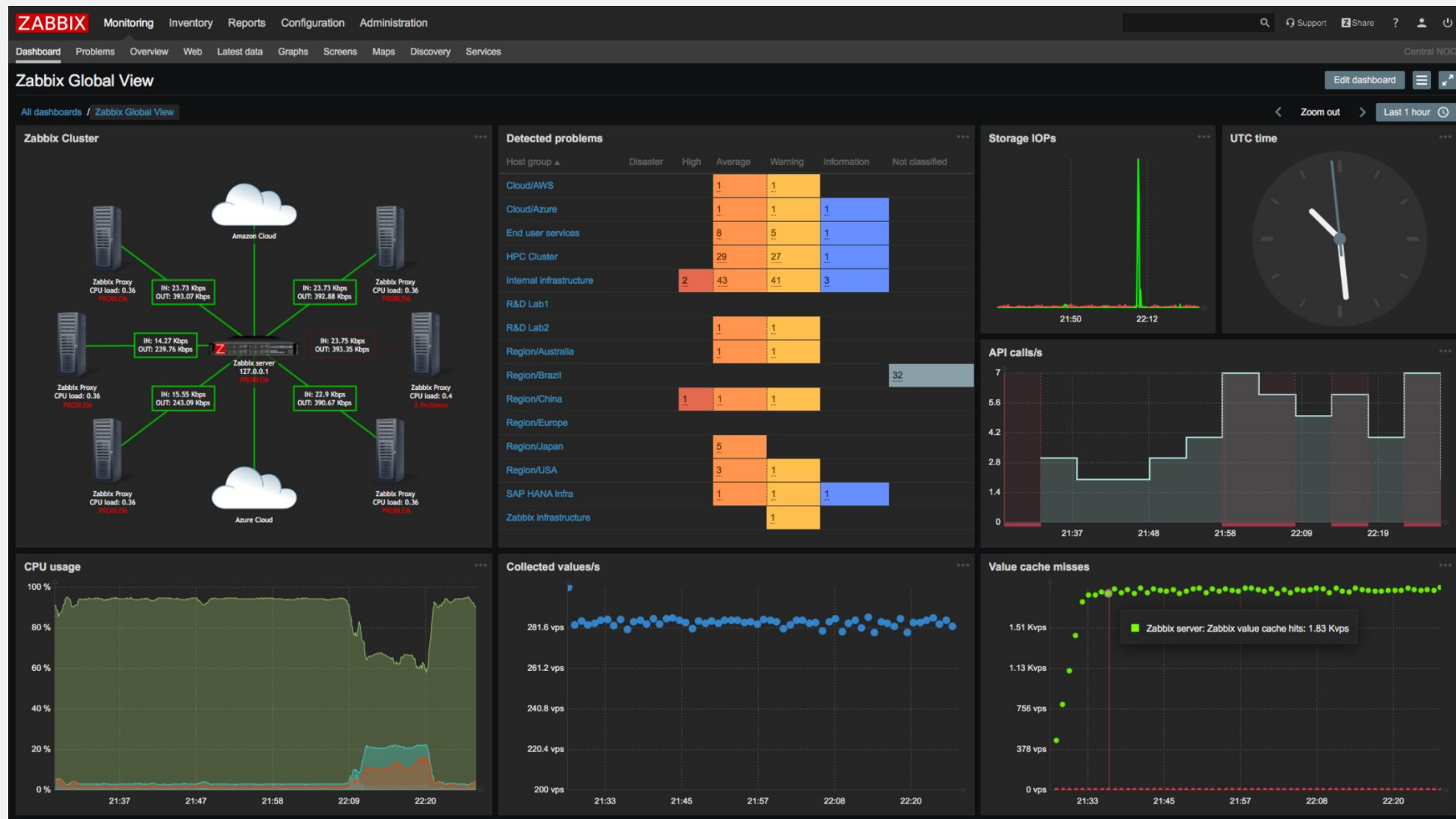
开源数据可视化工具：Grafana



第三节 人工智能平台开发流程

人工智能平台交付与运维

开源数据可视化工具：Zabbix



04 人工智能平台应用实战

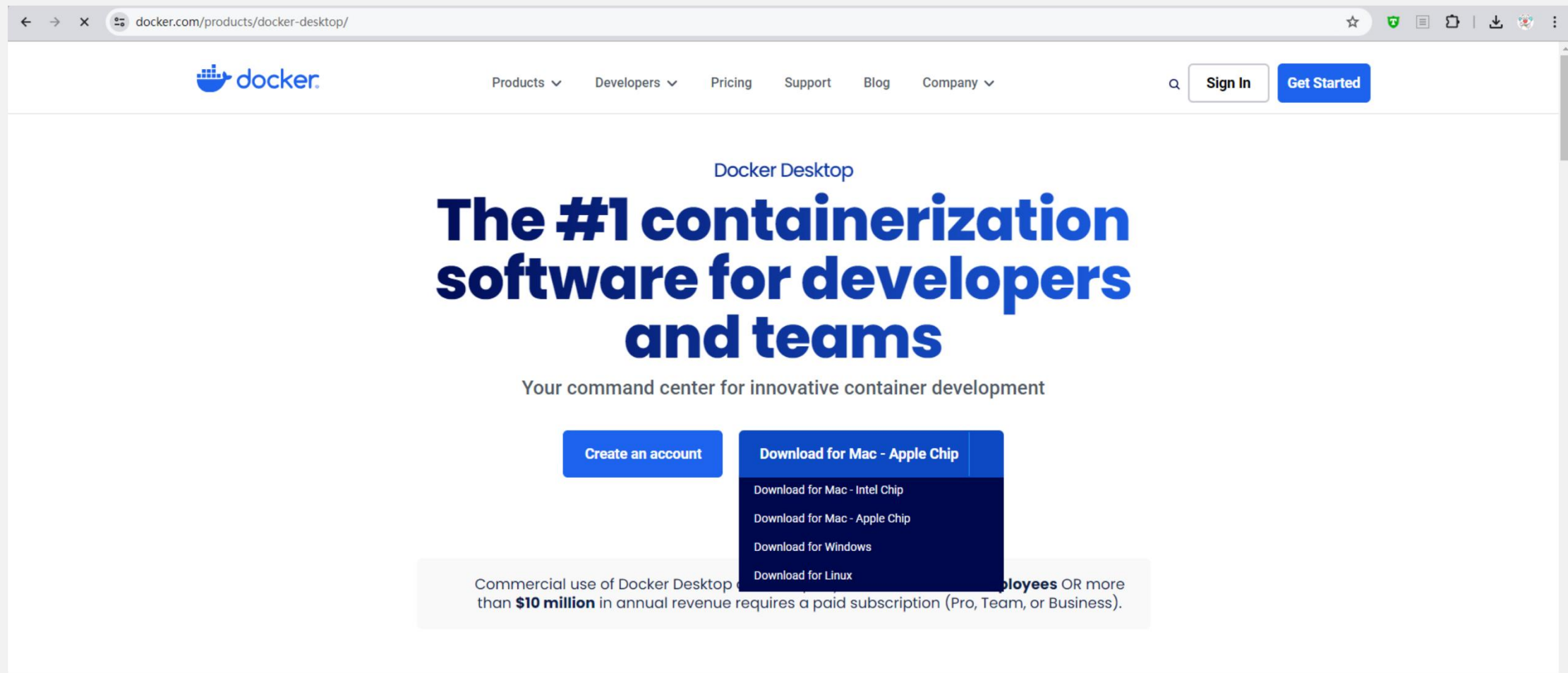
- 容器化部署：Docker的安装使用
- 案例：TrWebOcr-离线图像识别系统

第四节 人工智能平台应用实战

Docker安装使用

<https://www.docker.com/products/docker-desktop/>

官网下载Docker桌面版



第四节 人工智能平台应用实战

Docker安装使用

<https://docs.docker.com/desktop/install/ubuntu/>

Ubuntu操作系统安装docker指南

The screenshot shows the Docker Docs website with the navigation menu on the left and the main content area. The page title is "Install Docker Desktop on Ubuntu". The left sidebar lists various guides, with "Install on Ubuntu" selected. The main content area includes a "Docker Desktop terms" section, a "DEB package" button, a "Prerequisites" section, and a code block for installing gnome-terminal.

docs.docker.com/desktop/install/ubuntu/

docker:docs Guides Manuals Reference Samples FAQ

Manuals / Docker Desktop / Install Docker Desktop / Installation per Linux distro / Install on Ubuntu

Install Docker Desktop on Ubuntu

1 Docker Desktop terms

Commercial use of Docker Desktop in larger enterprises (more than 250 employees OR more than \$10 million USD in annual revenue) requires a [paid subscription](#).

This page contains information on how to install, launch and upgrade Docker Desktop on an Ubuntu distribution.

[DEB package](#)

For checksums, see [Release notes](#)

Prerequisites

To install Docker Desktop successfully, you must:

- Meet the [system requirements](#)
- Have a 64-bit version of either the latest LTS version (Ubuntu Jammy Jellyfish 22.04) or the current non-LTS version (Ubuntu Mantic Minotaur 23.10). Docker Desktop is supported on `x86_64` (or `amd64`) architecture.
- For non-Gnome Desktop environments, `gnome-terminal` must be installed:

```
$ sudo apt install gnome-terminal
```

Contents

Prerequisites

- Install Docker Desktop
- Launch Docker Desktop
- Upgrade Docker Desktop
- Next steps

Give feedback

第四节 人工智能平台应用实战

Docker安装使用

来源：菜鸟教程

<https://www.runoob.com/docker/docker-tutorial.html>

Docker Hello World

Docker 允许你在容器内运行应用程序，使用 **docker run** 命令来在容器内运行一个应用程序。

输出Hello world

```
runoob@runoob:~$ docker run ubuntu:15.10 /bin/echo "Hello world"
Hello world
```

```
runoob@runoob:~$ docker run ubuntu:15.10 /bin/echo "Hello world"
Hello world
```

各个参数解析：

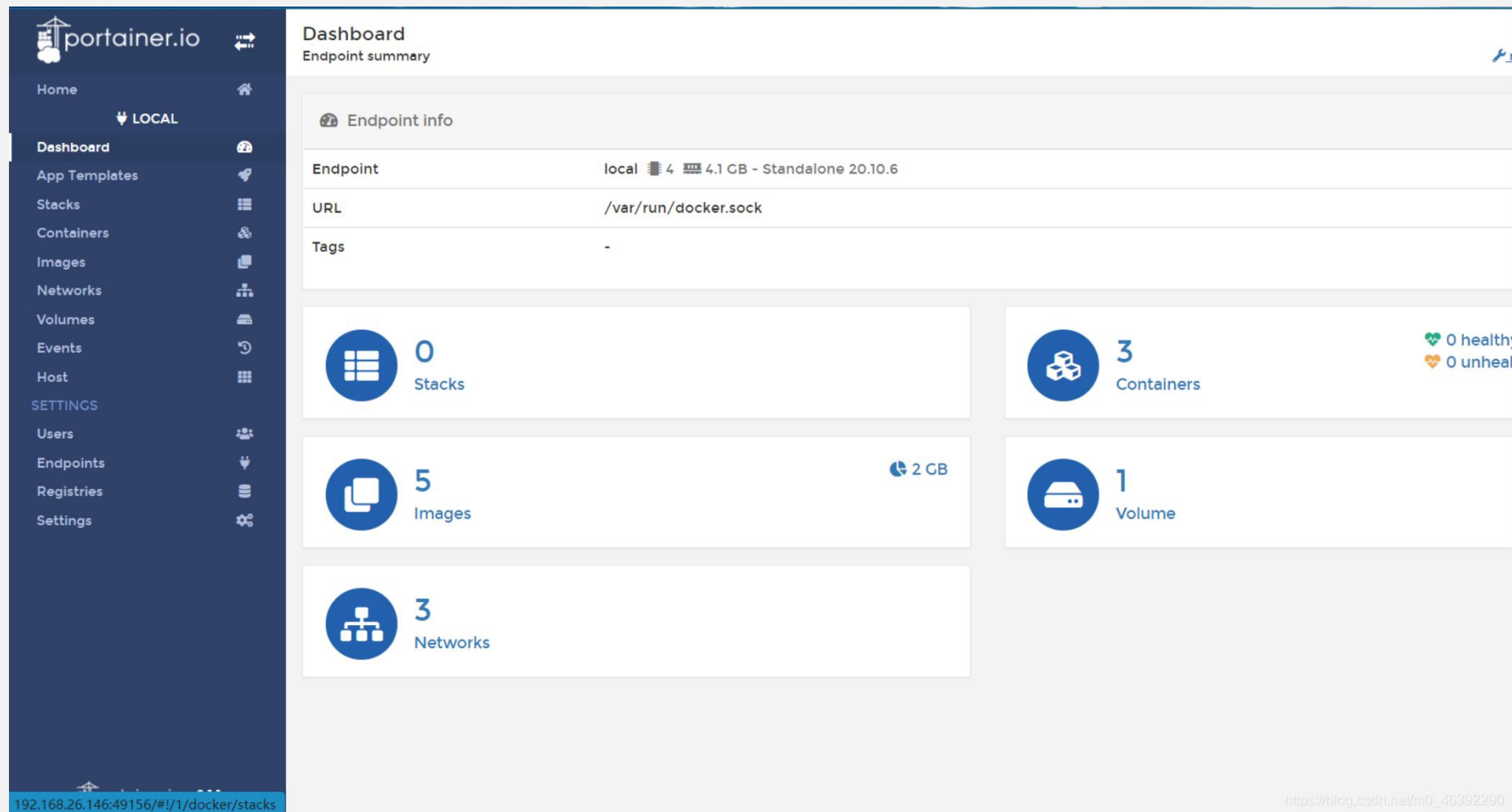
- **docker:** Docker 的二进制执行文件。
- **run:** 与前面的 docker 组合来运行一个容器。
- **ubuntu:15.10** 指定要运行的镜像，Docker 首先从本地主机上查找镜像是否存在，如果不存在，Docker 就会从镜像仓库 Docker Hub 下载公共镜像。
- **/bin/echo "Hello world":** 在启动的容器里执行的命令

以上命令完整的意思可以解释为：Docker 以 ubuntu15.10 镜像创建一个新容器，然后在容器里执行 bin/echo "Hello world"，然后输出结果。

第四节 人工智能平台应用实战

Docker-Portainer

<https://docs.portainer.io/start/install-ce/server/docker/linux>



- 容器管理
- 镜像管理
- 网络管理
- 数据卷管理
- 用户与权限管理

第四节 人工智能平台应用实战

Git安装使用

<https://gitforwindows.org/>

Windows操作系统安装Git



第四节 人工智能平台应用实战

Git安装使用

来源：菜鸟教程

<https://www.runoob.com/git/git-install-setup.html>

Linux 平台上安装

Git 的工作需要调用 curl, zlib, openssl, expat, libiconv 等库的代码，所以需要先安装这些依赖工具。

在有 yum 的系统上（比如 Fedora）或者有 apt-get 的系统上（比如 Debian 体系），可以用下面的命令安装：

各 Linux 系统可以使用其安装包管理工具（apt-get、yum 等）进行安装：

Debian/Ubuntu

Debian/Ubuntu Git 安装命令为：

```
$ apt-get install libcurl4-gnutls-dev libexpat1-dev gettext \
libz-dev libssl-dev

$ apt-get install git

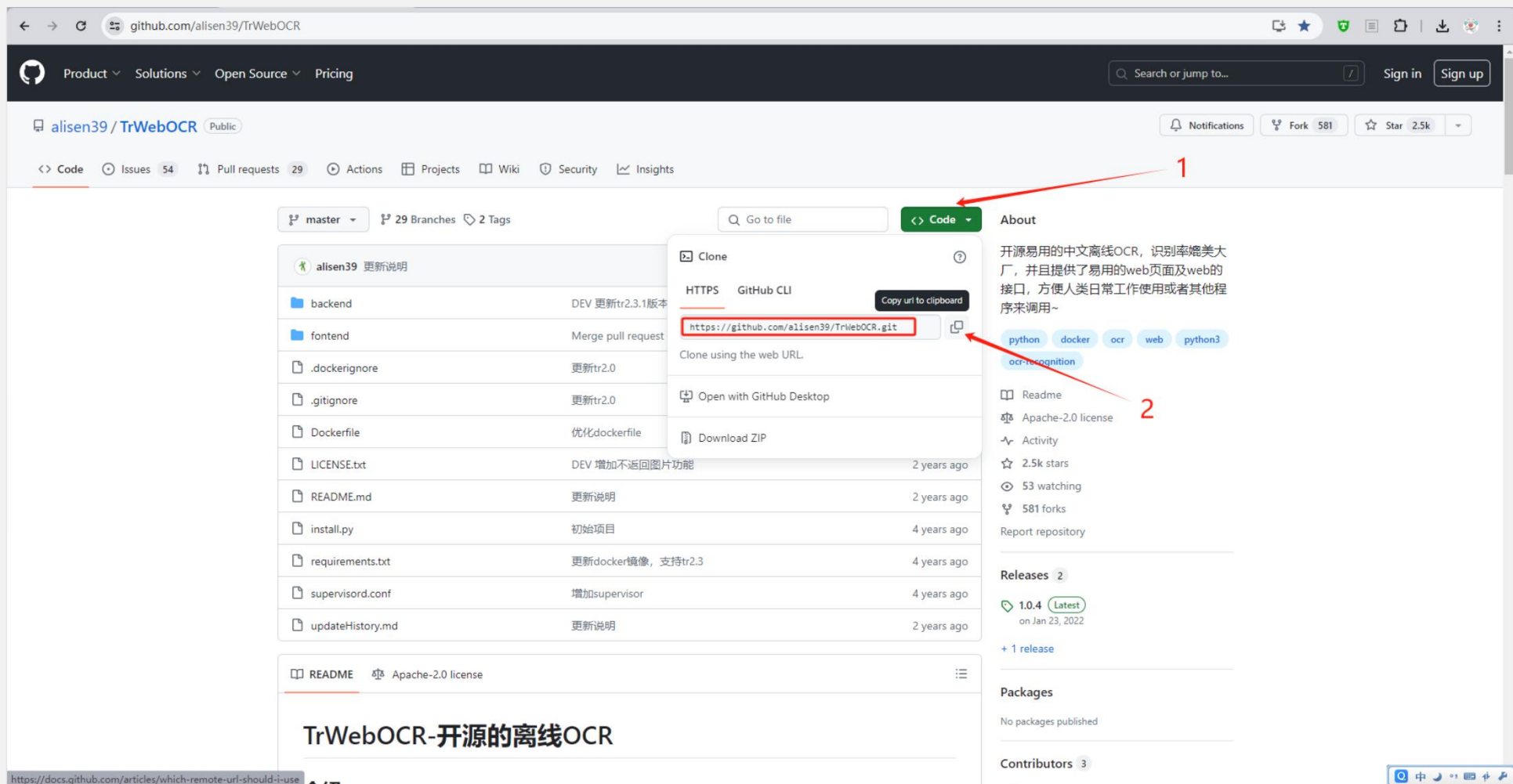
$ git --version
git version 1.8.1.2
```


第四节 人工智能平台应用实战

TrWebOCR

<https://github.com/alisen39/TrWebOCR>

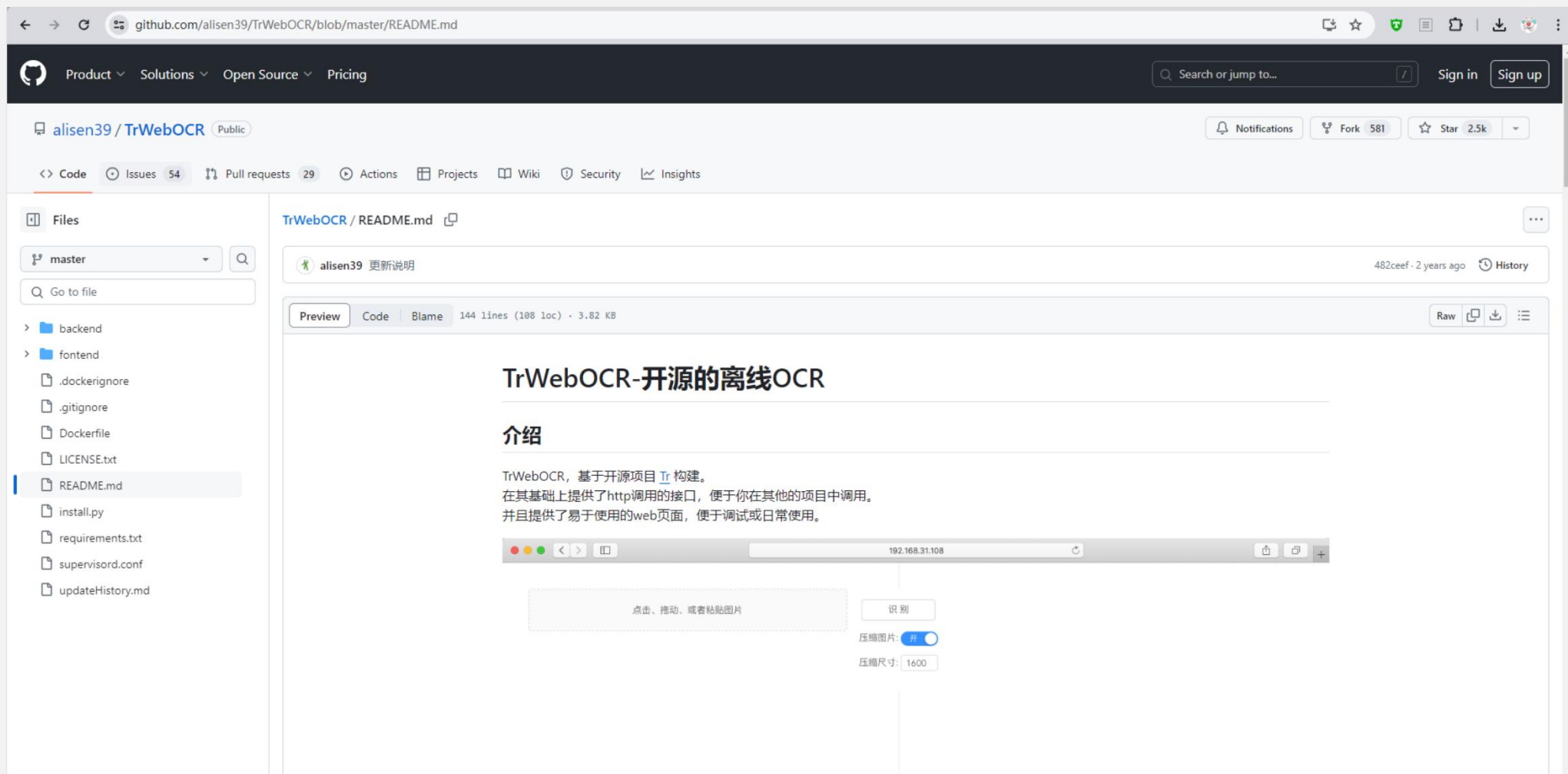
```
git clone https://github.com/alisen39/TrWebOCR.git
```



第四节 人工智能平台应用实战

TrWebOCR

<https://github.com/alisen39/TrWebOCR/blob/master/README.md>



第四节 人工智能平台应用实战

TrWebOCR

<http://s.whty.com.cn/>

```
docker run -itd --rm -p 8089:8089 --name trwebocr mmmz/trwebocr:latest
```

TrWebOCR 好用开源的离线OCR (<https://github.com/alisen39/TrWebOCR>)

点击、拖动、或者粘贴图片

识别

压缩图片: ☒ 开

压缩尺寸: 1600

文字检测结果

原始结果

识别的文字

产品名称: 一体机电脑
TPC-I045-23
制造商: HP Inc.
总代理: 中国惠普有限公司
总代理地址: 北京市朝阳区广顺南大街08号院1号楼5层
生产厂: 英业达(重庆)有限公司
生产厂地址: 中国重庆市沙坪坝区西区二路66号
执行标准: GB4943.1-2011, GB/T9254-2008, GB17625.1-2012
纸箱尺寸: 长 66.0 cm x 宽 24.5 cm x 高 46.2 cm
毛重: 11.284 公斤
净重: 5.49 公斤
合格证

中国制造 RCC

一体机电脑 合格证

Packing List

装箱清单

PC System
Keyboard
Mouse
Quick Setup Booklet
Manuals Documentation
Power Cable
Adapter

PC系统
键盘
鼠标
安装指南
说明书及附件袋
电源线
电源供应器

产品名称: 一体机电脑
TPC-I045-23
制造商: HP Inc.
总代理: 中国惠普有限公司



感谢观看

